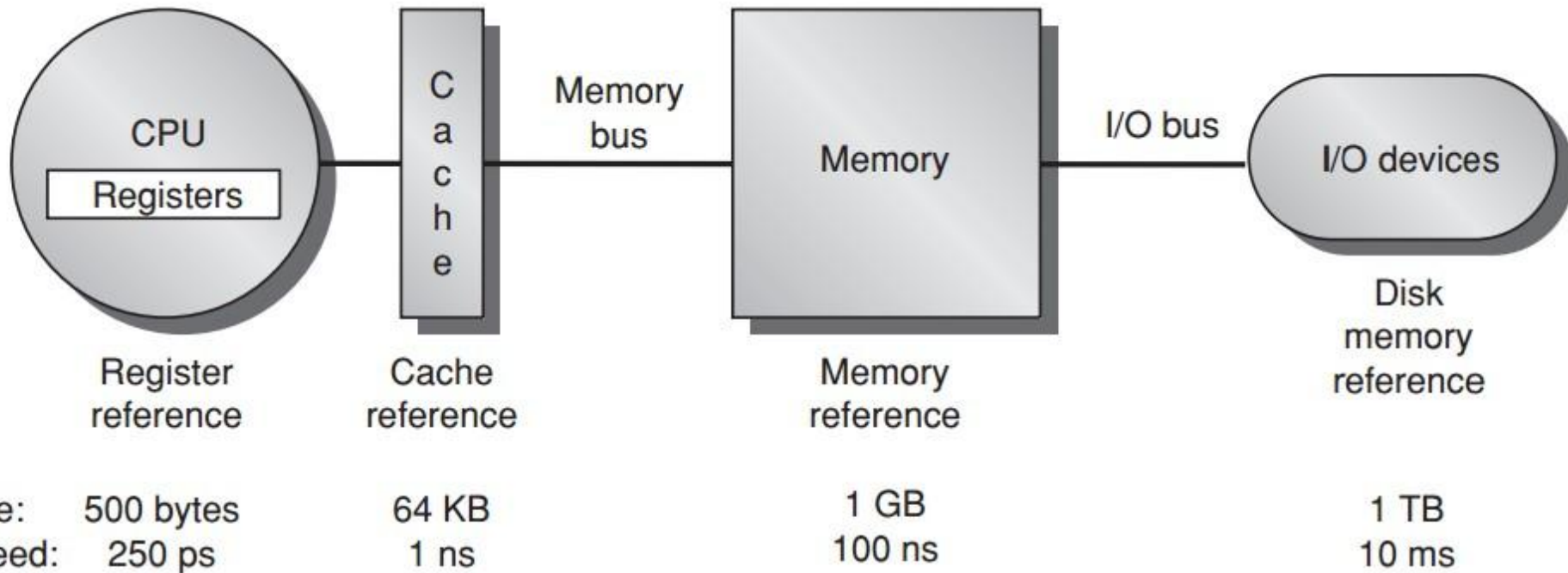


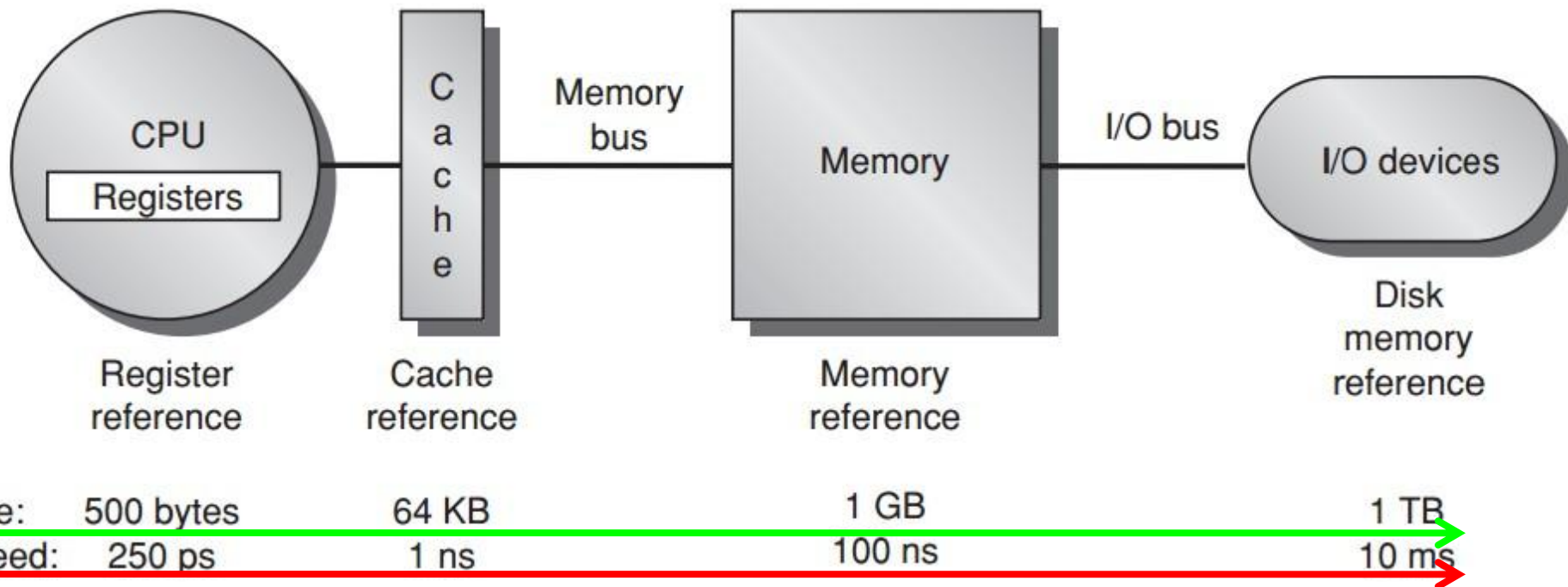
Memory Hierarchy Design Basics

Memory Hierarchy?

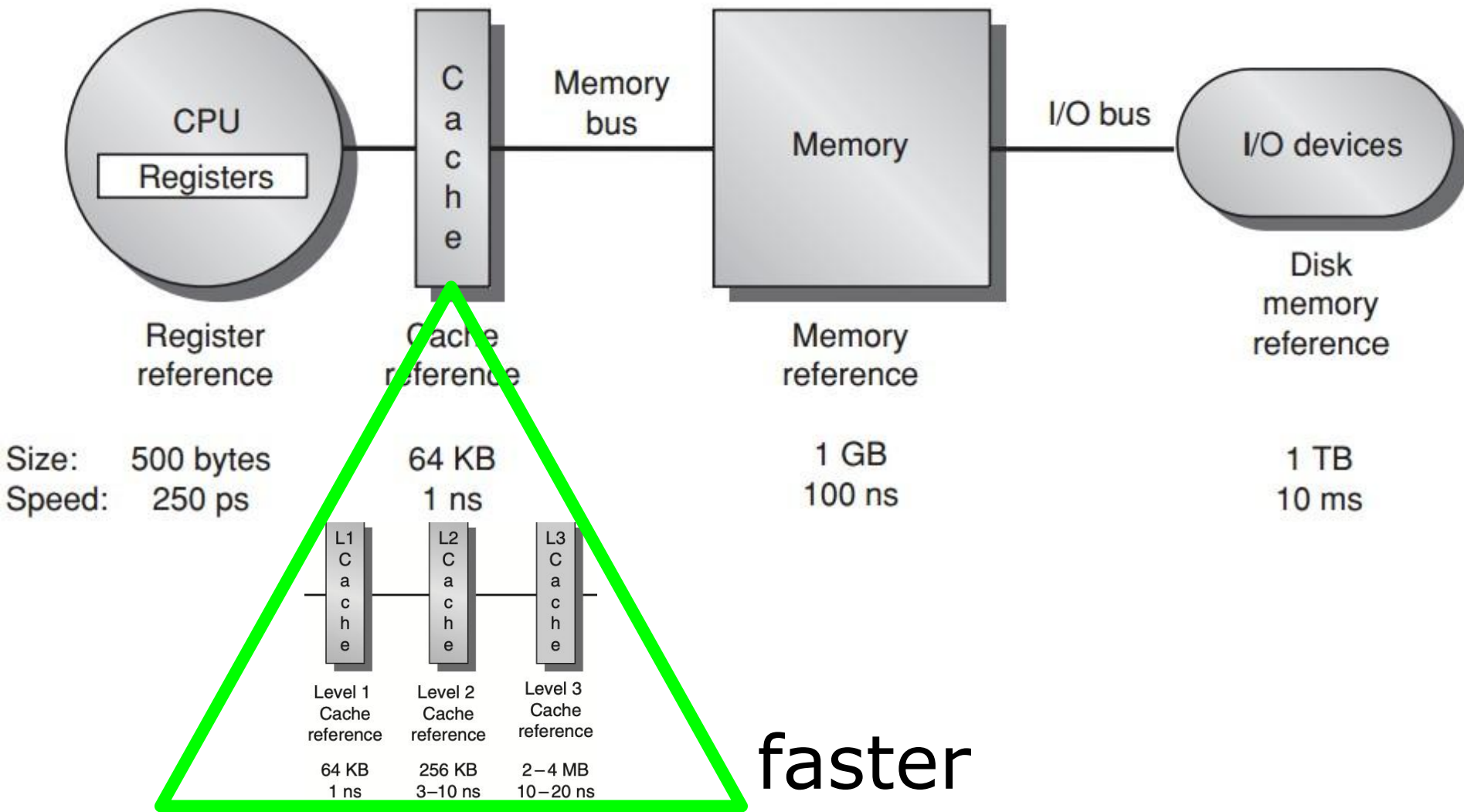
Memory Hierarchy



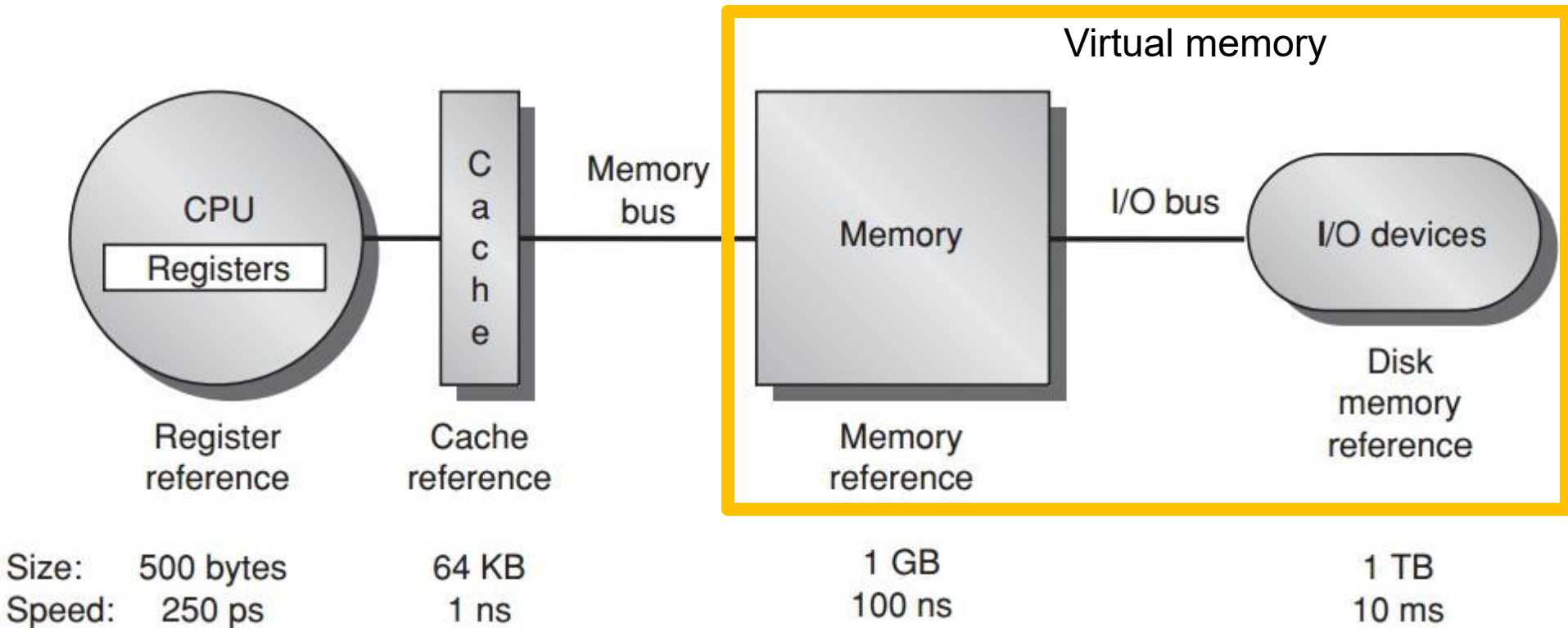
Memory Hierarchy



Memory Hierarchy

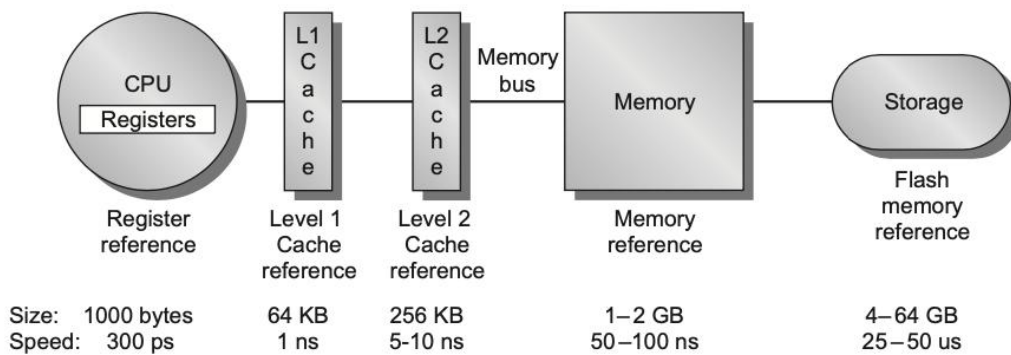


Memory Hierarchy

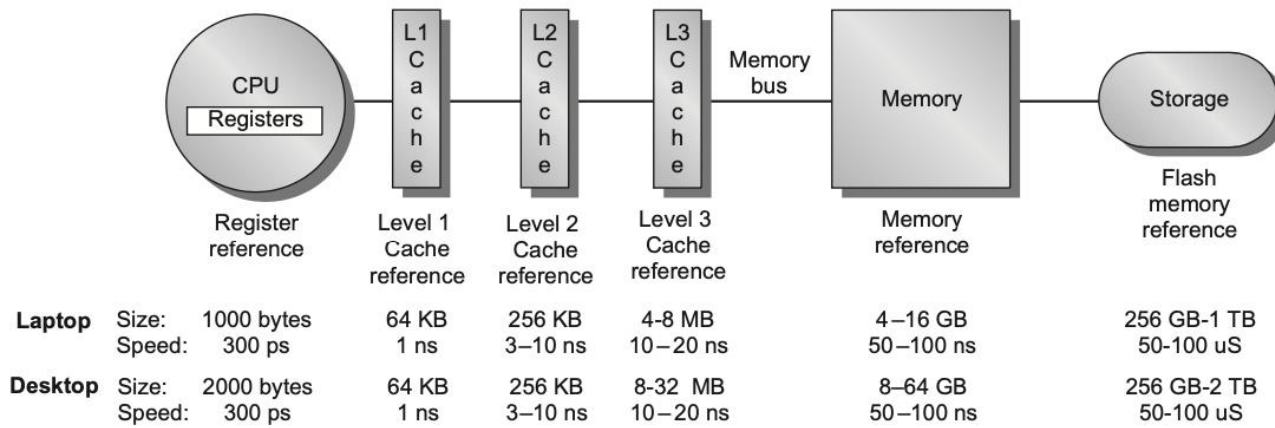


larger, safer

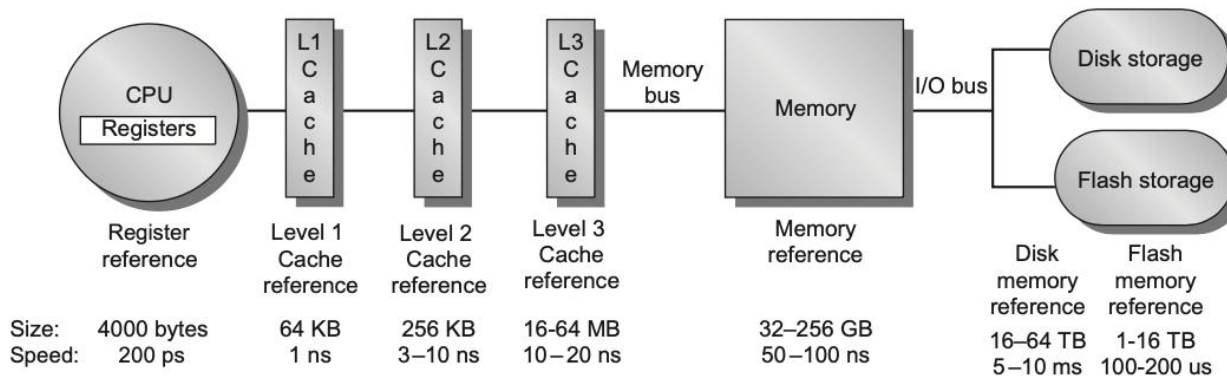
Memory Hierarchy



(A) Memory hierarchy for a personal mobile device

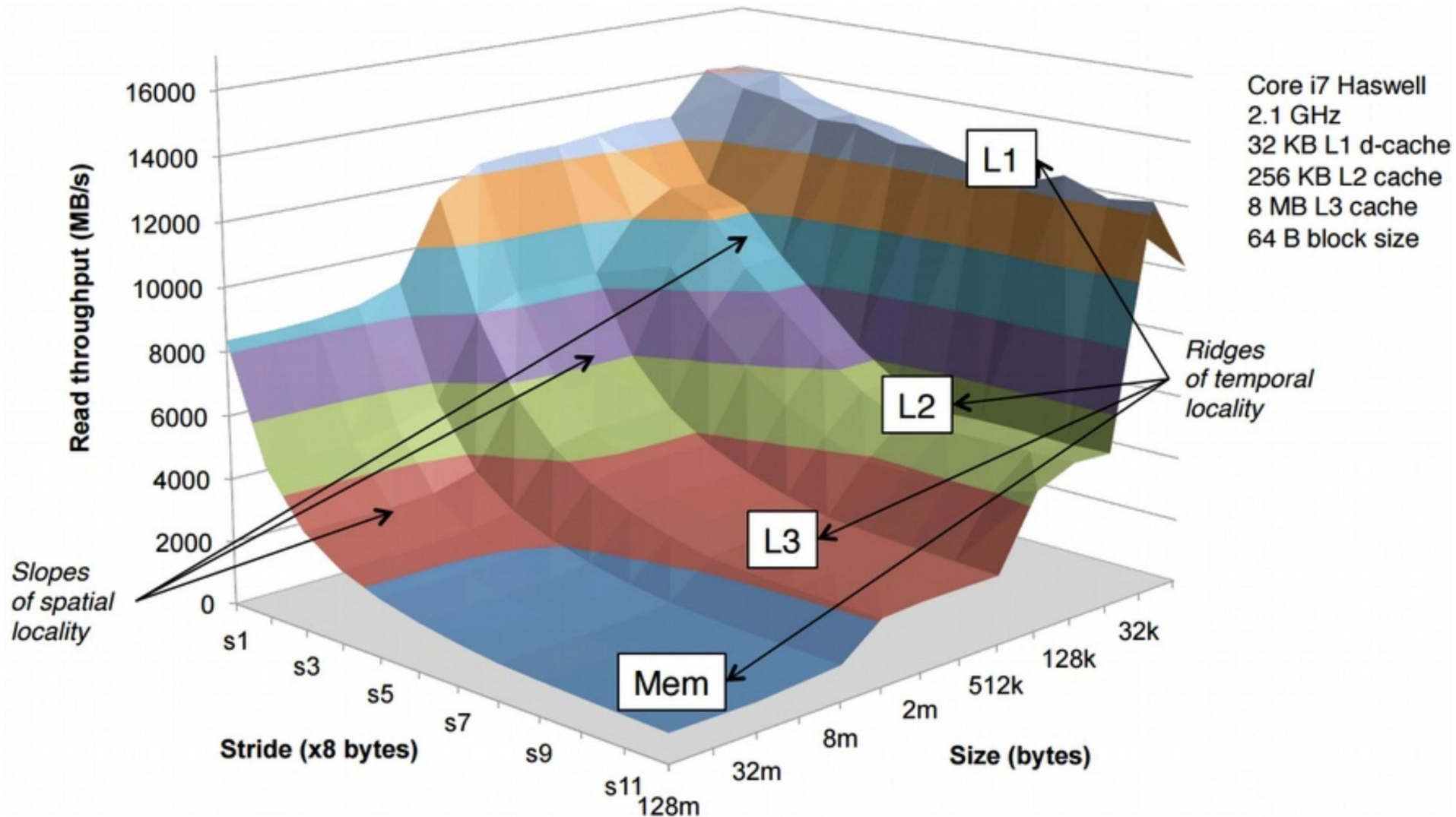


(B) Memory hierarchy for a laptop or a desktop



(C) Memory hierarchy for server

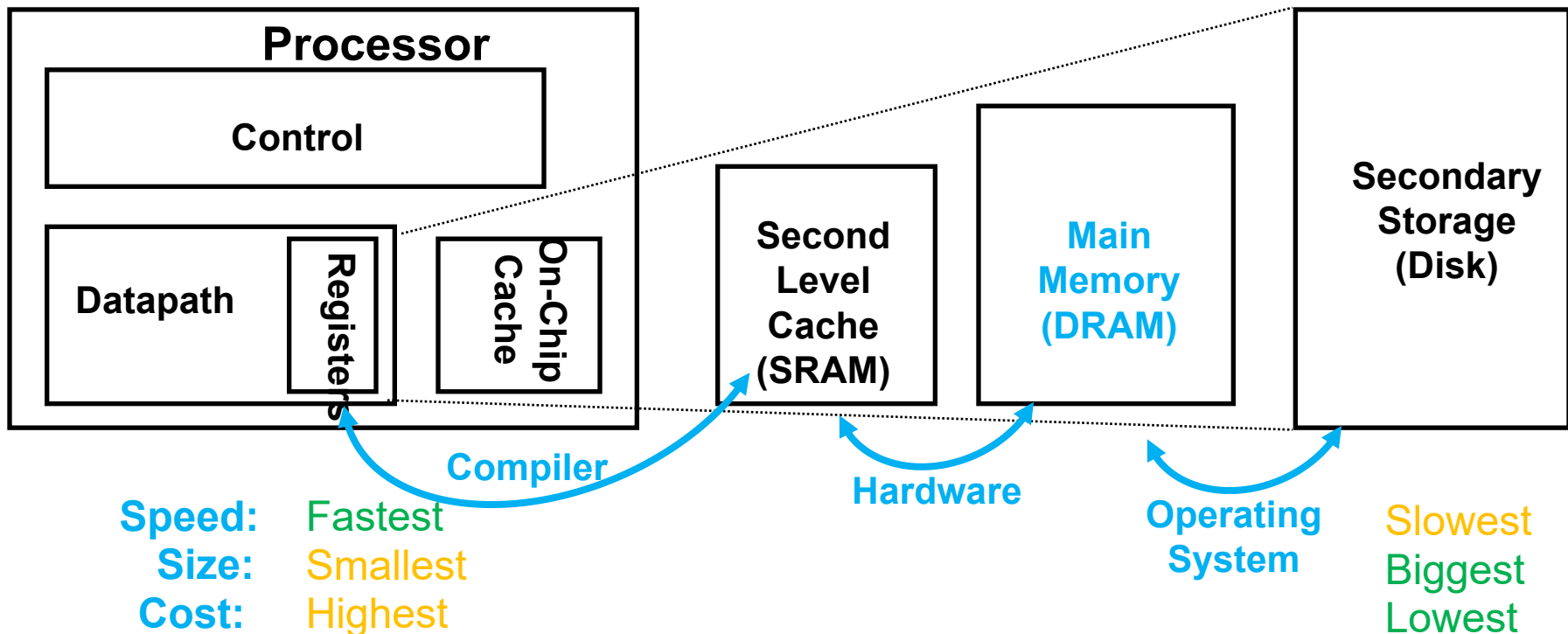
The Memory Mountain



Technology & Optimizations

Main Memory

- Main memory: I/O interface between caches and servers
- dst of input & src of output



Main Memory

Performance measures

- **Latency** the time to retrieve the first word of the block
important for caches;
harder to reduce;
- **Bandwidth** the time to retrieve the rest of this block
important for multiprocessors, I/O, and
caches with large block sizes;
easier to improve with new
organizations: e.g., multi-bank mem

Main Memory

Performance measures

- **Latency**

access time: the time between when a read is requested and when the desired word arrives;

cycle time: the minimum time between unrelated requests to memory;
or the minimum time between the start of an access and the start of the next access;

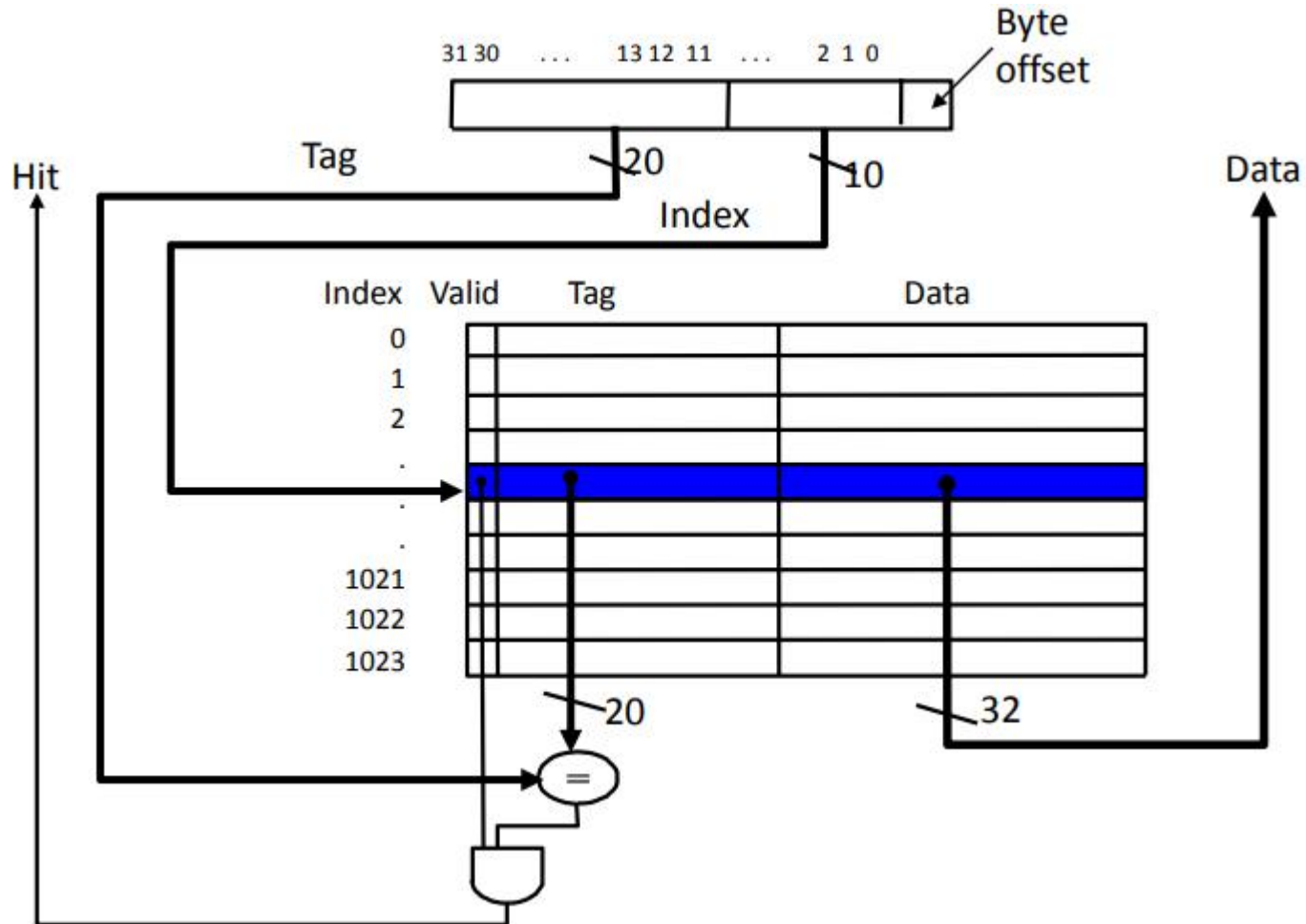
Main Memory

- SRAM for cache
- DRAM for main memory

SRAM

- Static Random Access Memory
- Six transistors per bit to prevent the information from being disturbed when read
- Don't need to refresh, so access time is very close to cycle time

Direct Mapped Cache



Direct Mapped Cache

- Example
 - 16 KB direct mapped cache
 - 256-byte blocks
- Which of the following addresses conflict with 0x12345678?
 - 0x12345677
 - 0x11335577
 - 0x11115678
 - 0x12341666

Direct Mapped Cache

- Example
 - 16 KB direct mapped cache
 - 256-byte blocks
- Which of the following addresses **conflict** with 0x12345678?
 - 0x12345677
 - 0x11335577 **be in diff blocks**
 - 0x11115678 **map to same cache lines**
 - 0x12341666

Direct Mapped Cache

- Example
16 KB direct mapped cache
256-byte blocks
- Which of the following addresses conflict with 0x12345678?
0x12345677
0x11335577 $\log_2 256 = 8$ -bit offset
0x11115678
0x12341666

Direct Mapped Cache

- Example

16 KB direct mapped cache

256-byte blocks

- Which of the following addresses conflict with 0x12345678?

0x12345677

0x11335577

0x11115678

0x12341666

part of...

$\log(16\text{KB}/256\text{B}) = 64$ lines

$\log 64 = 6$ -bit line index

Direct Mapped Cache

- Example
16 KB direct mapped cache
256-byte blocks
- Which of the following addresses conflict with 0x12345678?

0x12345677 ...01010110...

0x11335577 ...01010101...

0x11115678 ...01010110...

0x12341666 ...00010110...

part of...

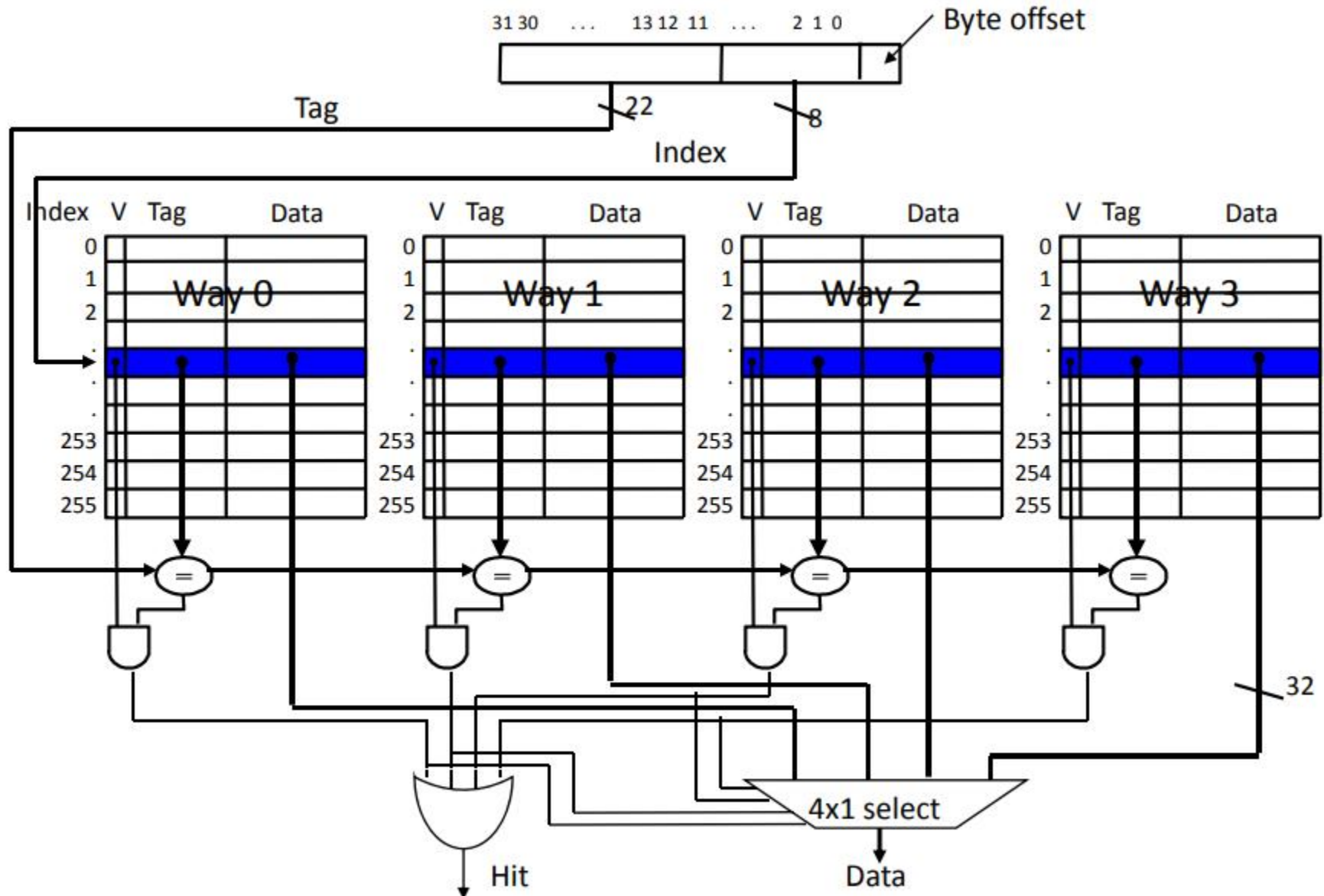
Direct Mapped Cache

- Example
16 KB direct mapped cache
256-byte blocks
- Which of the following addresses conflict with 0x12345678?

0x1234	56	77	...	01	01	01	10	...	same block
0x1133	55	77	...	01	01	01	01	...	diff line
0x1111	56	78	...	01	01	01	10	...	conflict
0x1234	16	66	...	00	01	01	10	...	conflict

part of...

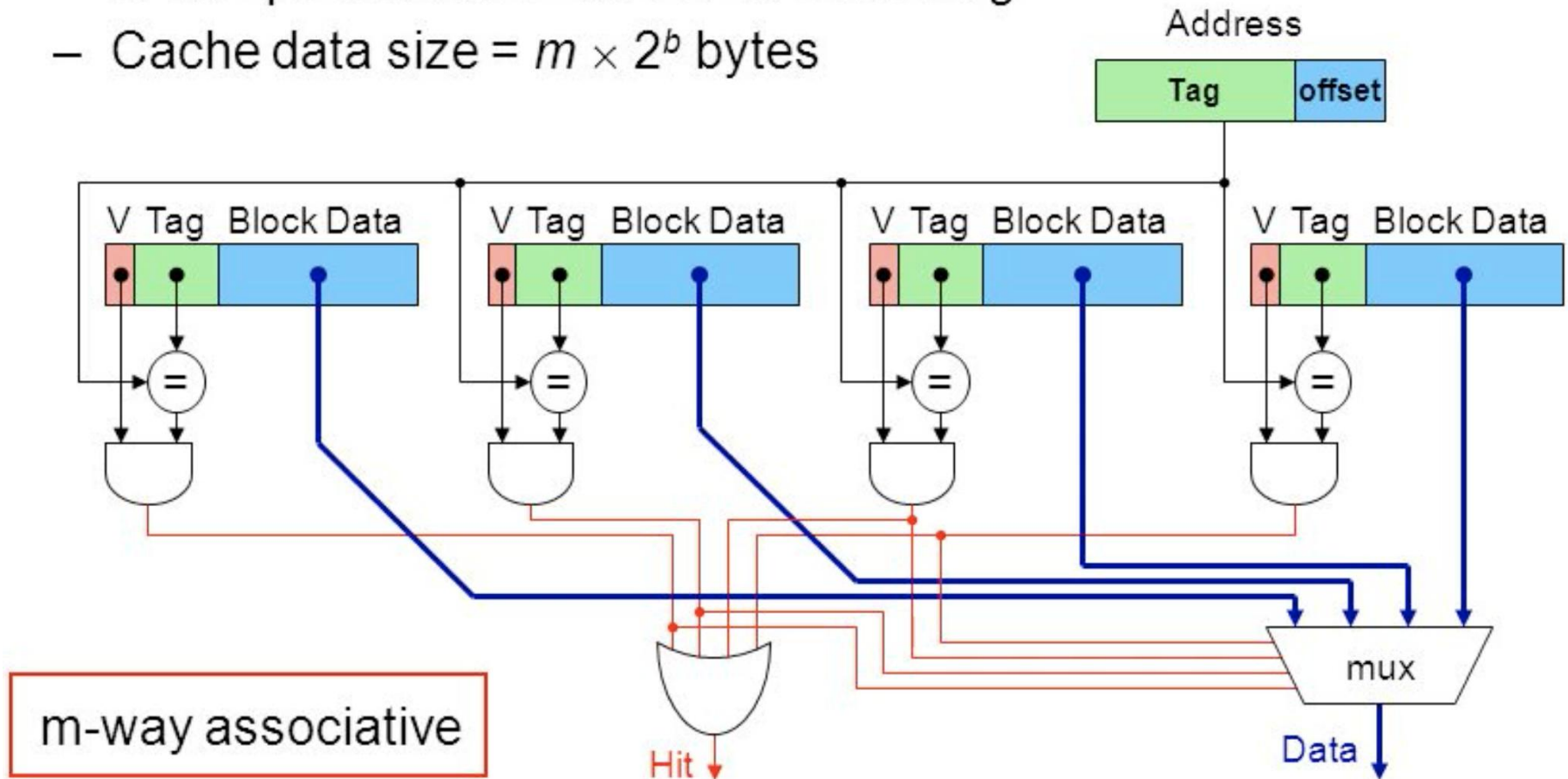
Set Associative Cache



Fully Associative Cache

If m blocks exist then

- m comparators are needed to match *tag*
- Cache data size = $m \times 2^b$ bytes



DRAM

- Dynamic Random Access Memory
- Single transistor per bit
- Reading destroys the information
- Refresh periodically
- cycle time > access time

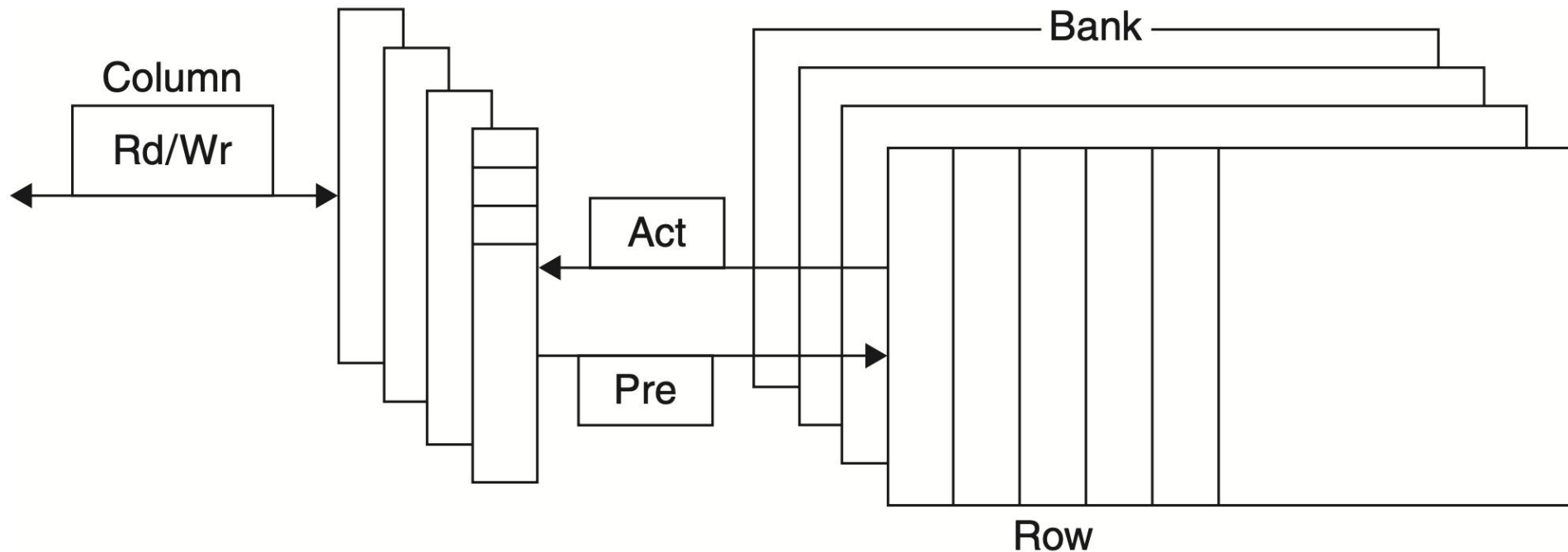
DRAM

- Dynamic Random Access Memory
- Single transistor per bit
- Reading destroys the information
- Refresh periodically
- cycle time > access time

DRAMs are commonly sold on small boards called **DIMM** (dual inline memory modules), typically containing 4 ~ 16 DRAMs

DRAM Organization

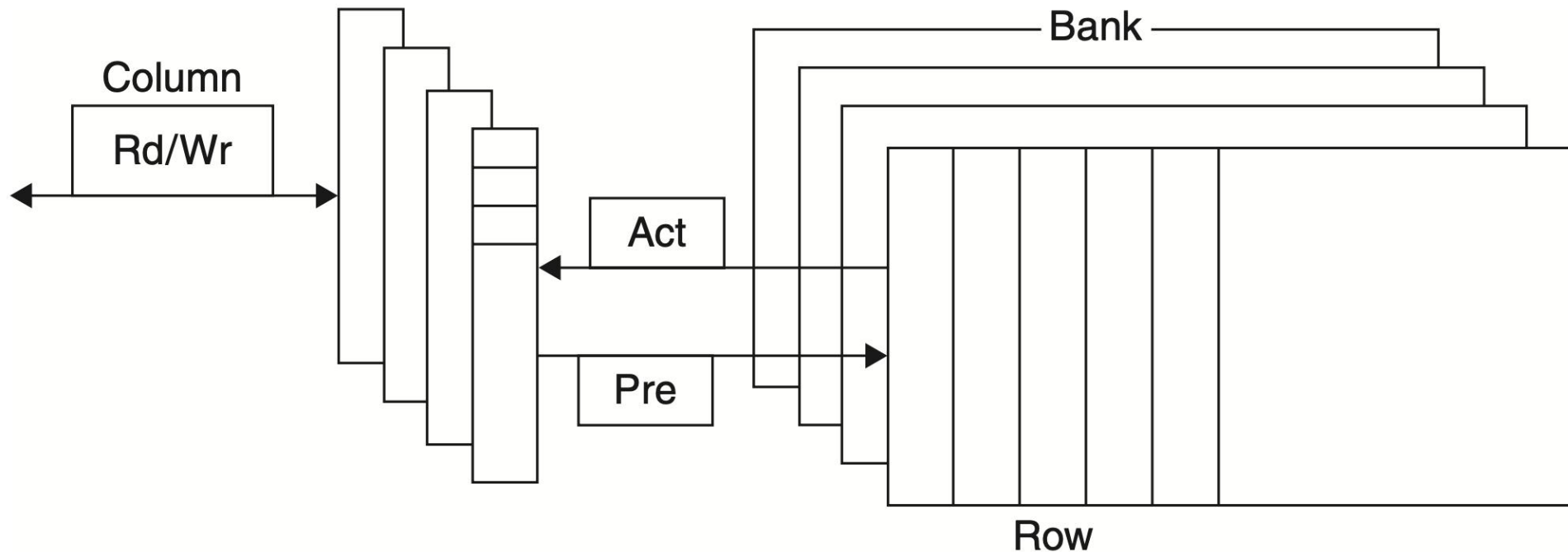
DRAMs are organized in banks



DRAM Organization

Each bank consists of a series of rows

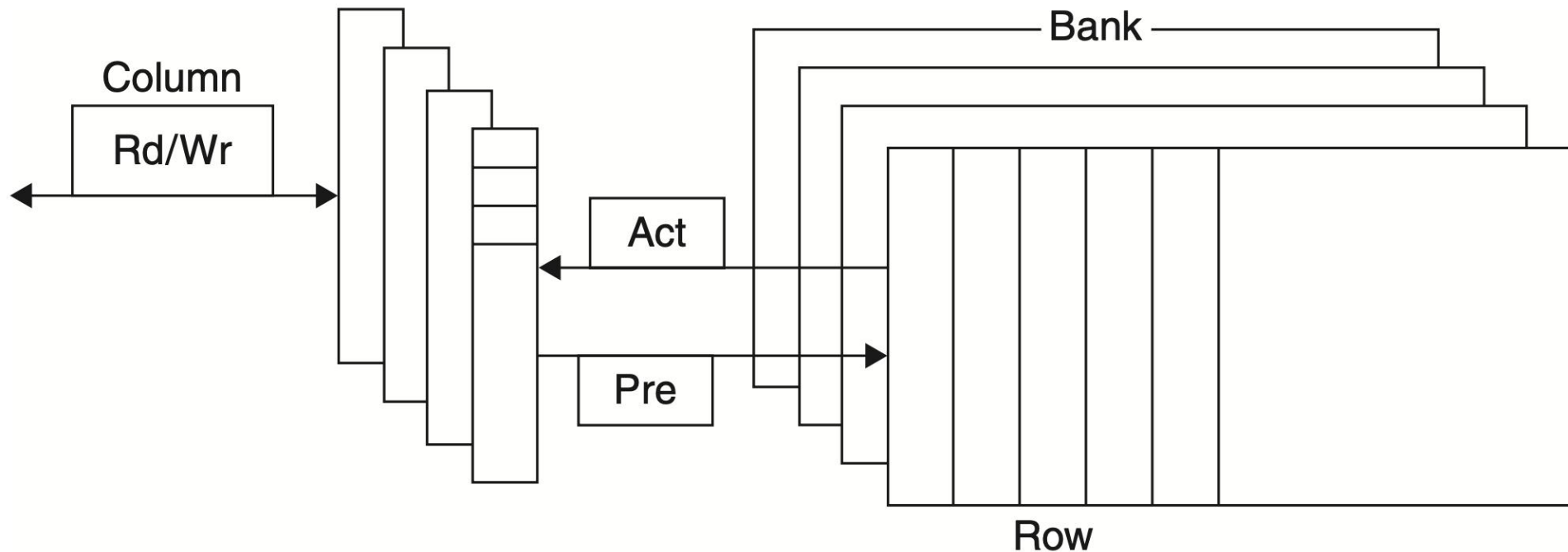
RAS: row access strobe



DRAM Organization

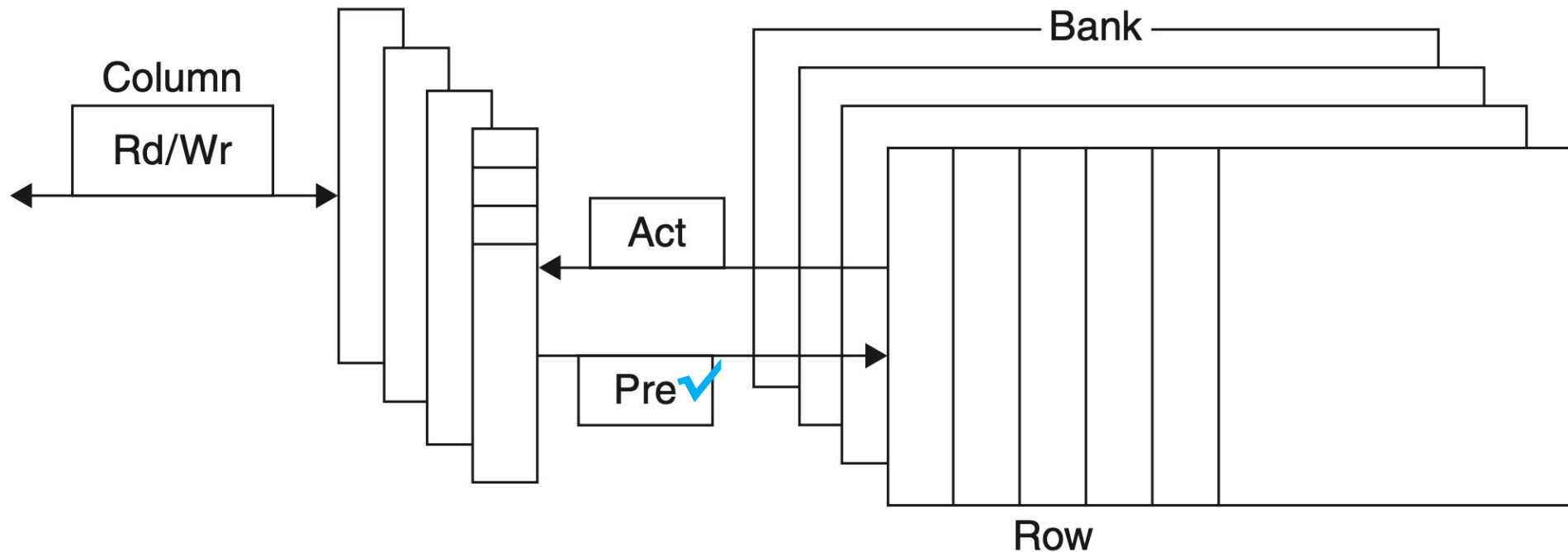
Each row consists of a series of columns

CAS: column access strobe



DRAM Organization

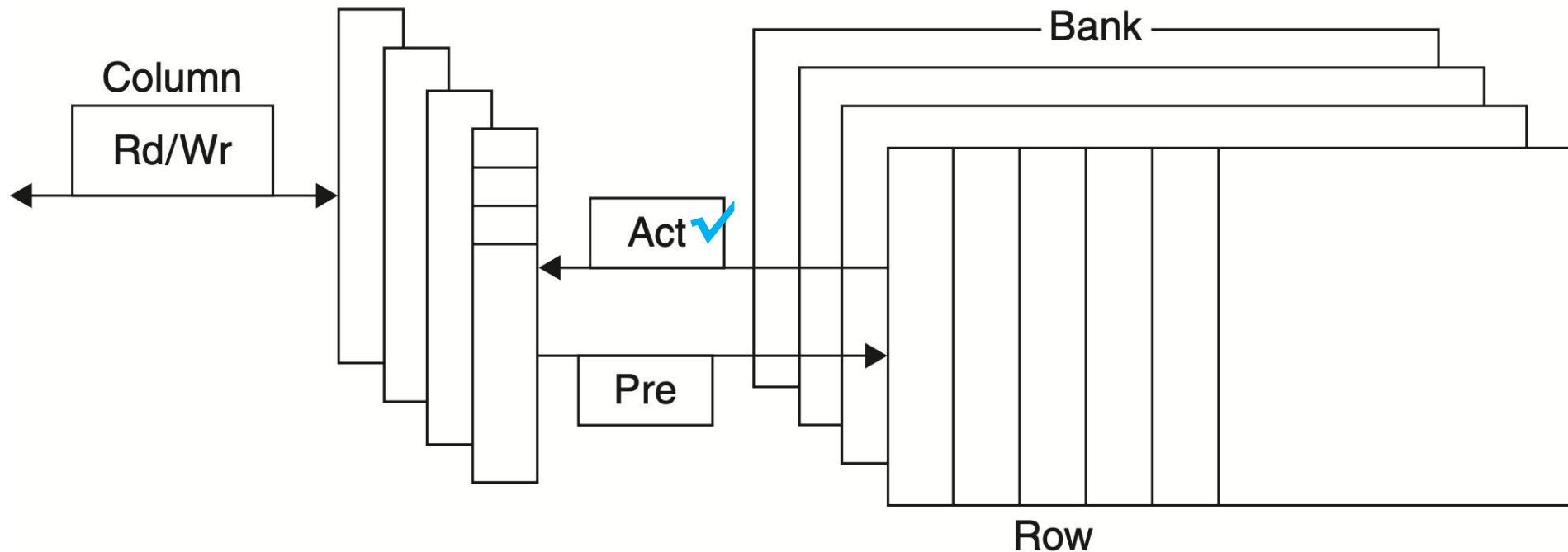
Pre (pre.charge) command opens or closes a bank



DRAM Organization

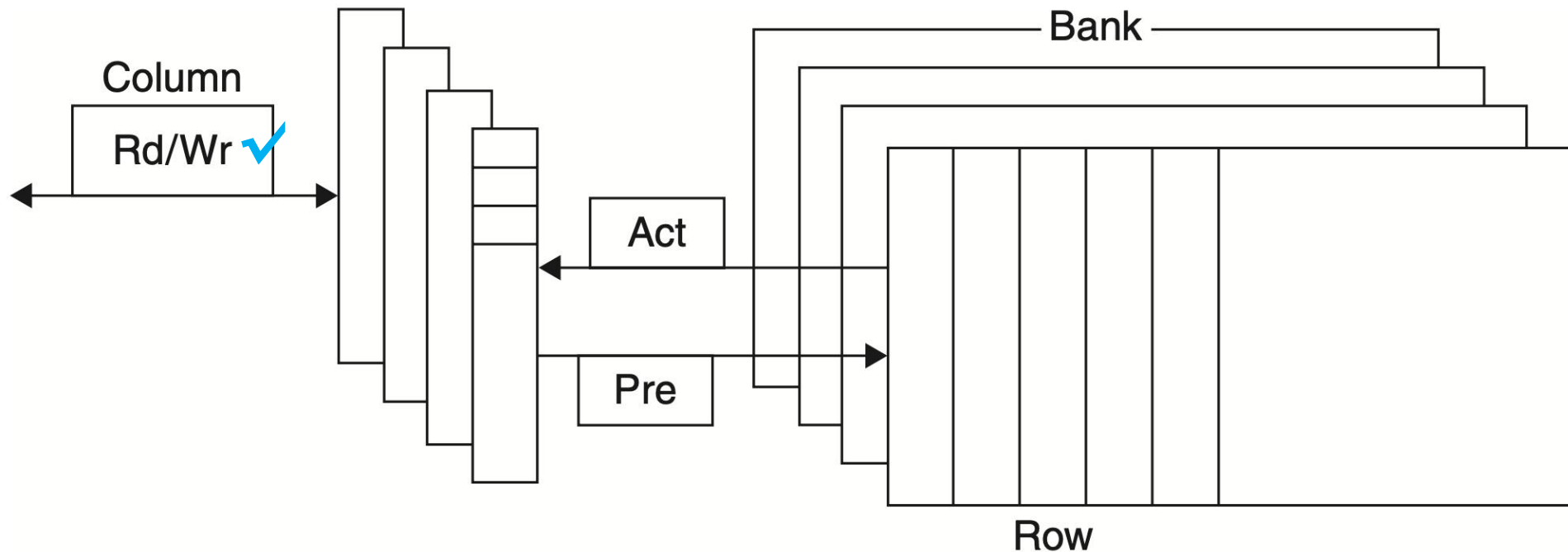
Row address is sent with an **Act (activate)** command;

Then row is transferred to a buffer

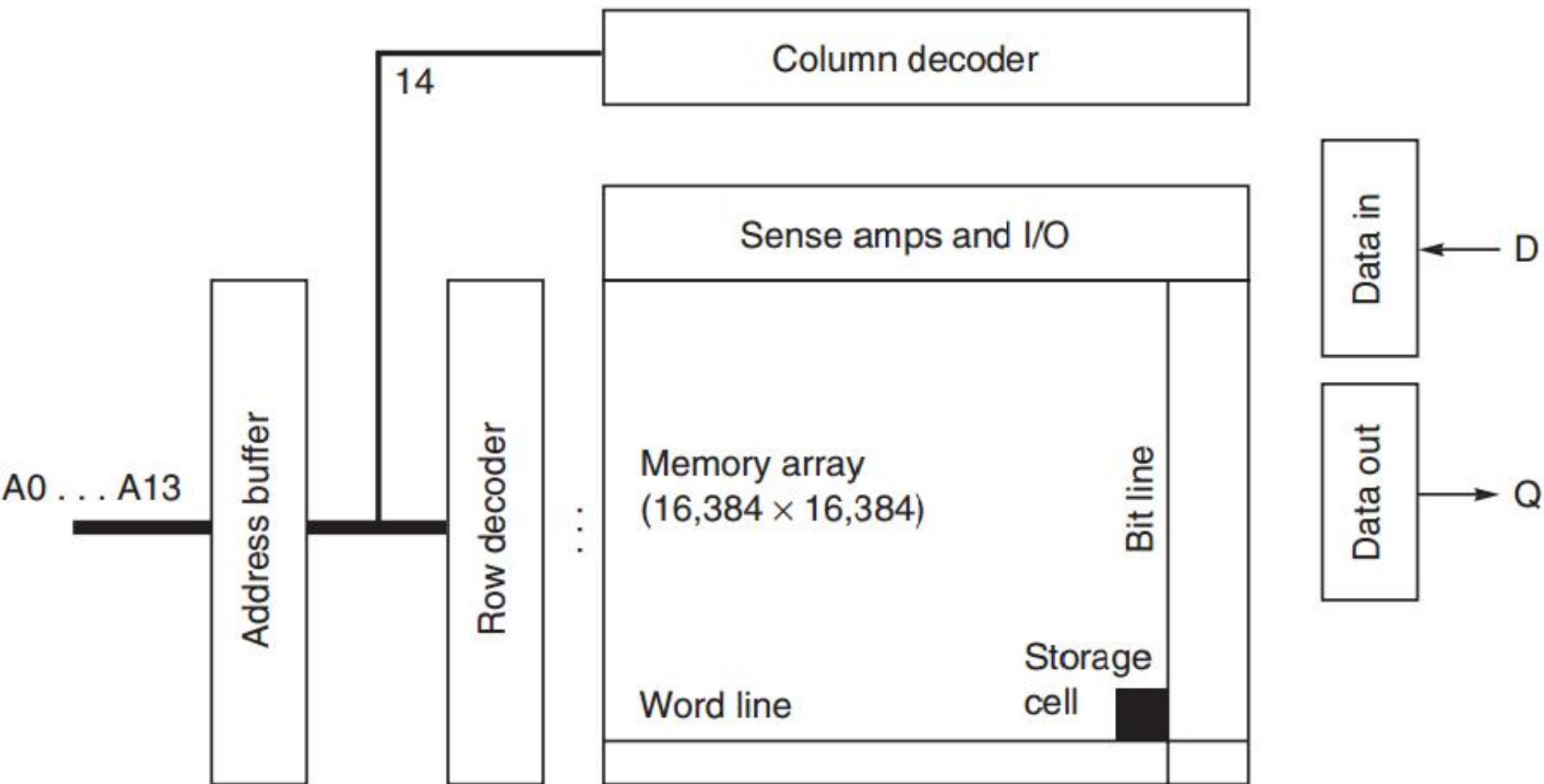


DRAM Organization

For row in the buffer, it can be transferred by successive column addresses

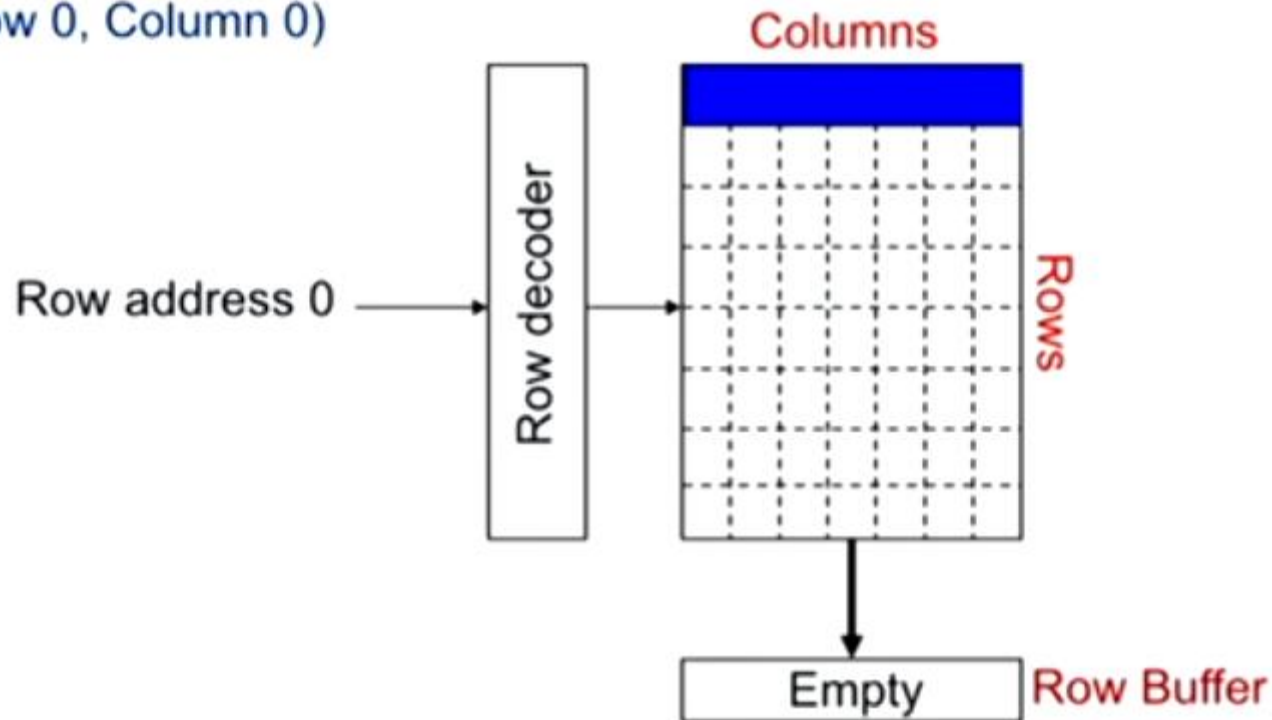


DRAM Organization



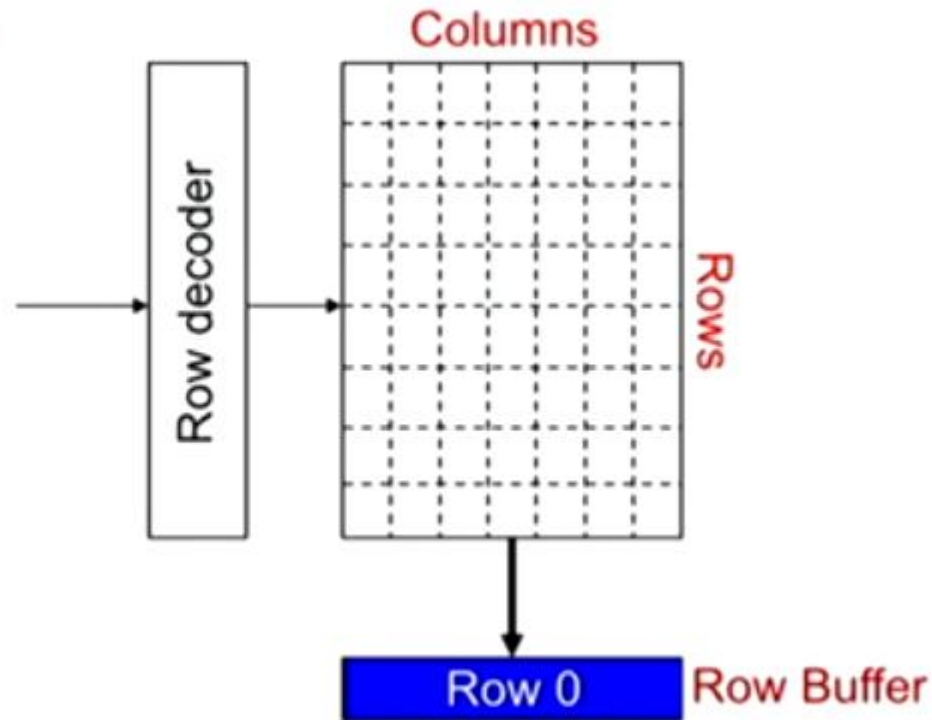
DRAM Access Example

Access Address:
(Row 0, Column 0)



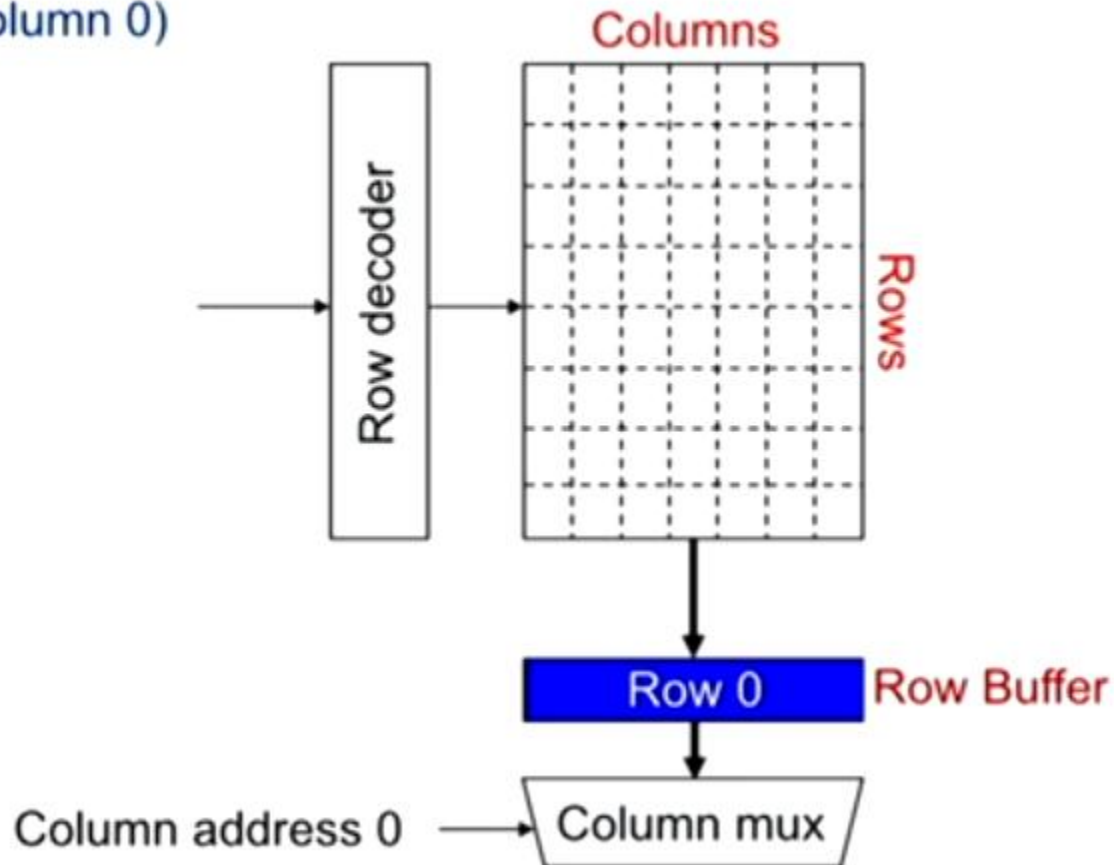
Bring Row into Row Buffer

Access Address:
(Row 0, Column 0)



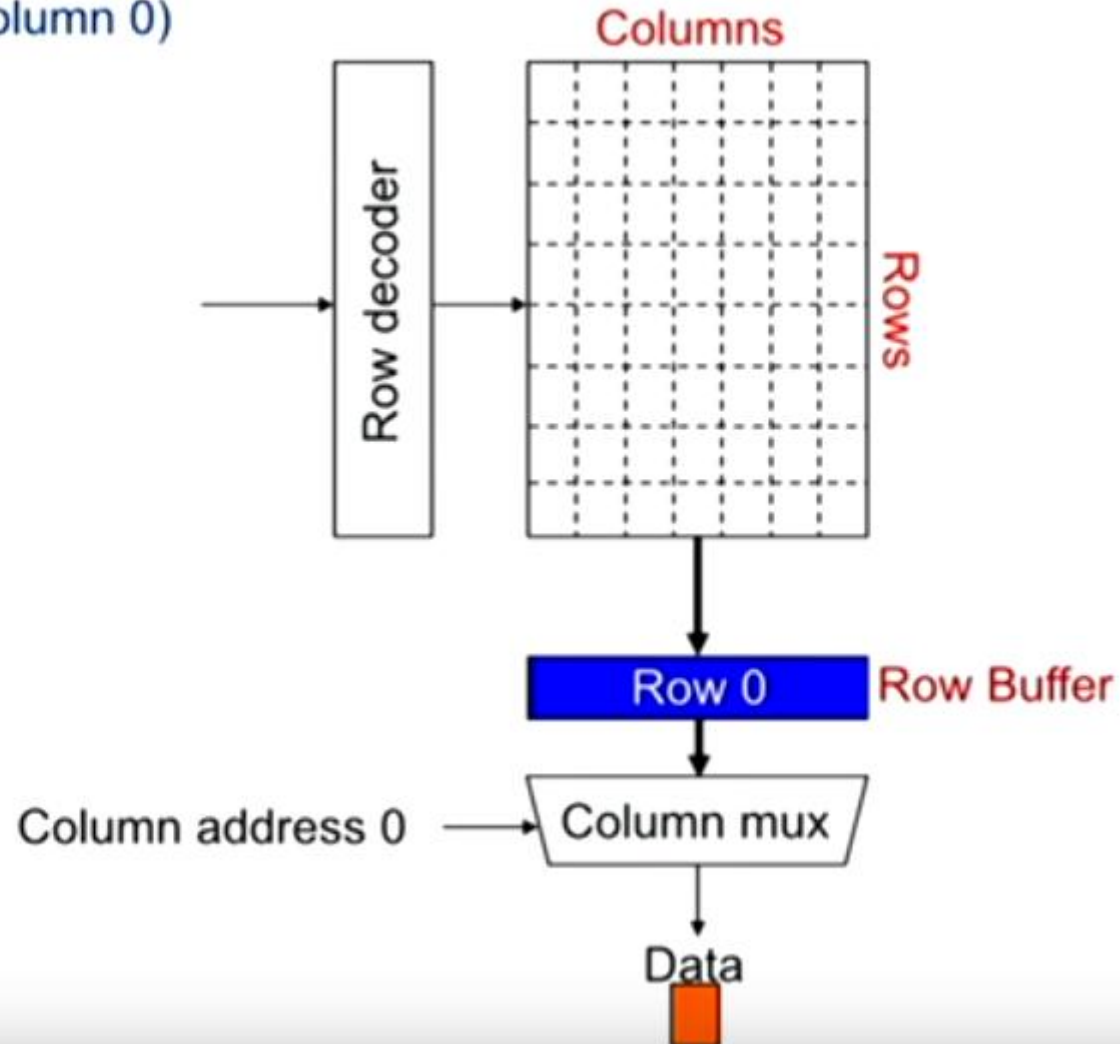
Select Data via Multiplexor

Access Address:
(Row 0, Column 0)



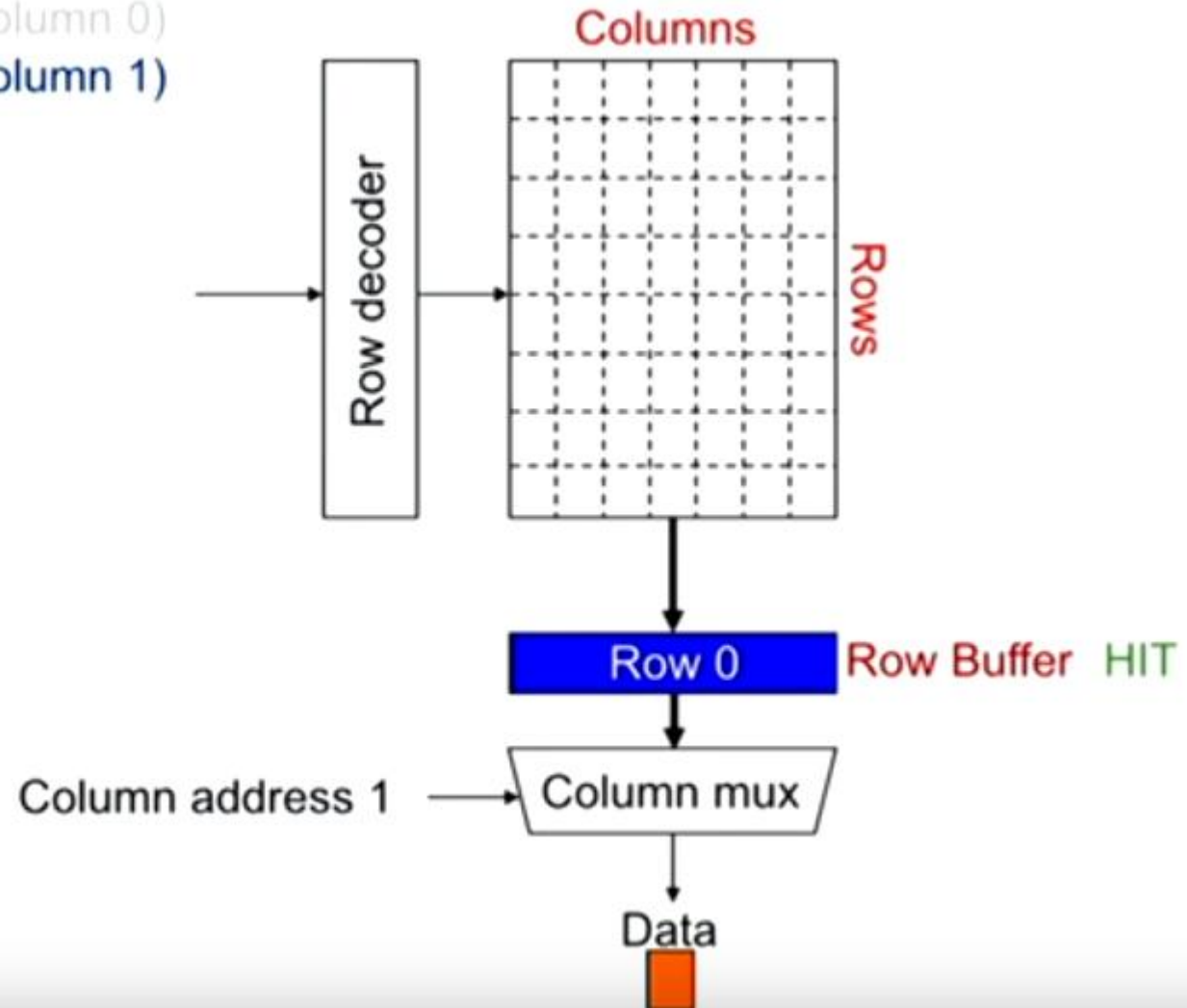
Data Selected

Access Address:
(Row 0, Column 0)



Row Buffer Hit

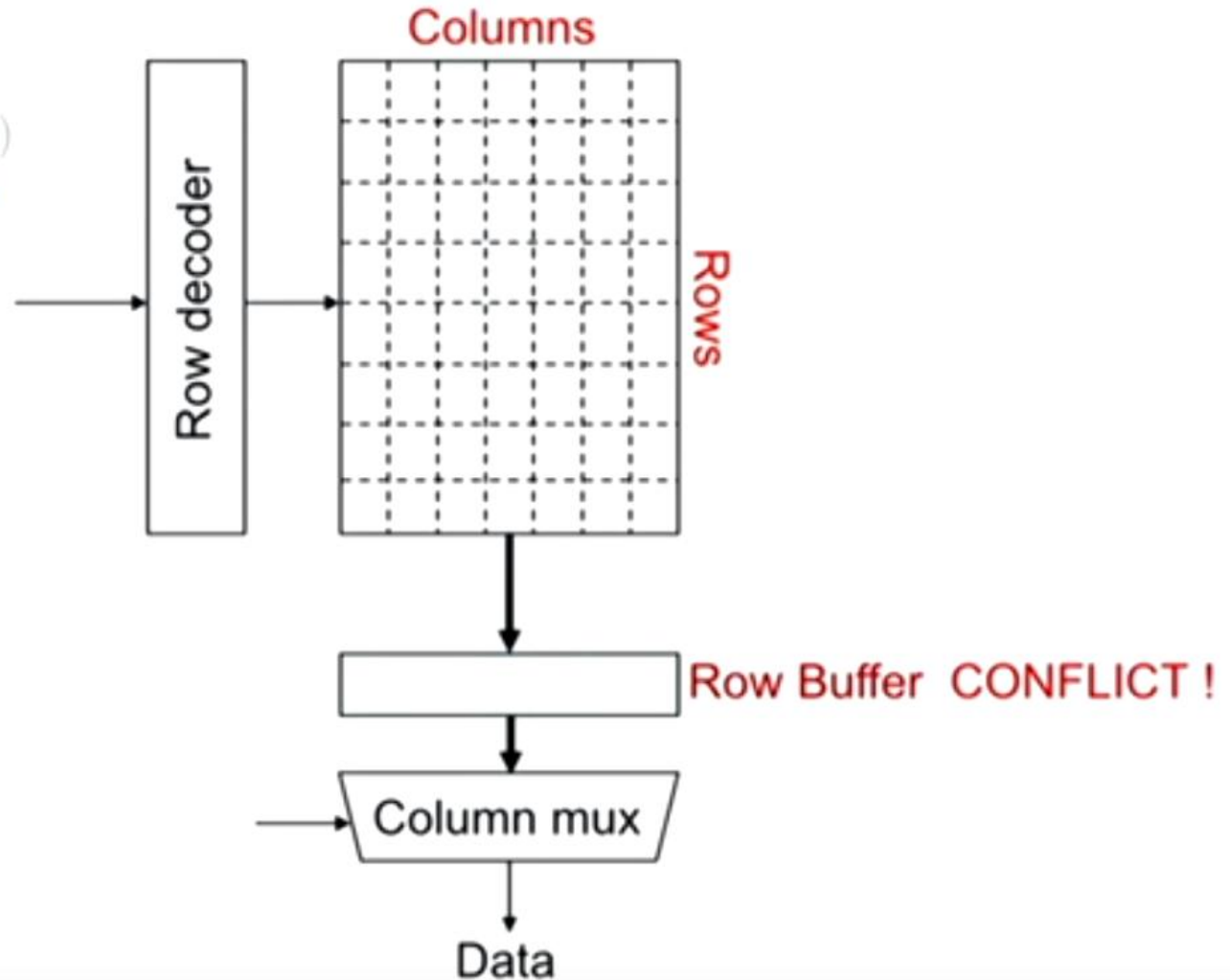
Access Address:
(Row 0, Column 0)
(Row 0, Column 1)



Row Buffer Conflict

Access Address:

(Row 0, Column 0)
(Row 0, Column 1)
(Row 0, Column 85)
(Row 1, Column 0)



DRAM Improvement

- **Timing signals**

allow repeated accesses to the row buffer w/o another row access time;

- **Leverage spatial locality**

each array will buffer 1024 to 4096 bits for each access;

DRAM Improvement

- **Clock signal**
added to the DRAM interface,
so that repeated transfers will not
involve overhead to synchronize with
memory controller;
- **SDRAM:** synchronous DRAM

DRAM Improvement

- **Wider DRAM**

widening memory for higher memory bandwidth;

e.g., 4-bit transfer mode up to 16-bit buses

DRAM Improvement

- **DDR: double data rate**

to increase bandwidth,
transfer data on both the rising edge
and falling edge of the DRAM clock
signal,
thereby doubling the peak data rate;

DRAM Improvement

- **Multiple Banks**

break a single SDRAM into 2~8 banks;
they can operate independently;

- Provide some of the advantages of interleaving
- Help with power management

DRAM Improvement

- **Reducing power consumption in SDRAMs**

dynamic power: used in read or write;
static/standby power;

- Depend on the operating voltage
- **Power down mode**: entered by telling the DRAM to ignore the clock
disables the SDRAM except for internal automatic refresh;

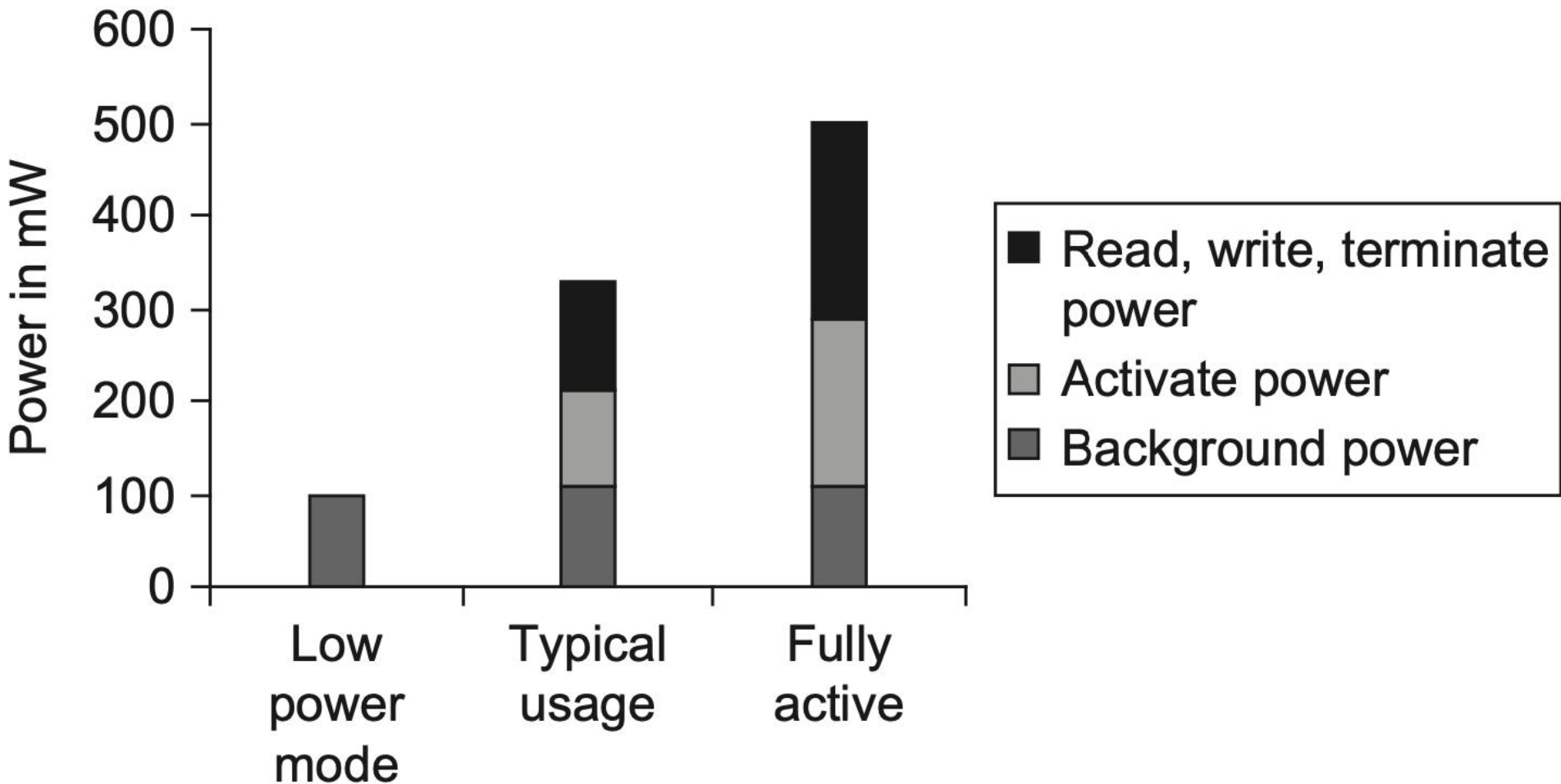
Capacity and Access Time

Production year	Chip size	DRAM type	Best case access time (no precharge)			Precharge needed
			RAS time (ns)	CAS time (ns)	Total (ns)	Total (ns)
2000	256M bit	DDR1	21	21	42	63
2002	512M bit	DDR1	15	15	30	45
2004	1G bit	DDR2	15	15	30	45
2006	2G bit	DDR2	10	10	20	30
2010	4G bit	DDR3	13	13	26	39
2016	8G bit	DDR4	13	13	26	39

Clock Rates and Bandwidth

Standard	I/O clock rate	M transfers/s	DRAM name	MiB/s/DIMM	DIMM name
DDR1	133	266	DDR266	2128	PC2100
DDR1	150	300	DDR300	2400	PC2400
DDR1	200	400	DDR400	3200	PC3200
DDR2	266	533	DDR2-533	4264	PC4300
DDR2	333	667	DDR2-667	5336	PC5300
DDR2	400	800	DDR2-800	6400	PC6400
DDR3	533	1066	DDR3-1066	8528	PC8500
DDR3	666	1333	DDR3-1333	10,664	PC10700
DDR3	800	1600	DDR3-1600	12,800	PC12800
DDR4	1333	2666	DDR4-2666	21,300	PC21300

Power Consumption



Graphics Data RAM

- GDRAM: Graphics DRAM
- GSDRAM: Graphics Synchronous DRAM
- Tailored for higher bandwidth demands of GPUs

Graphics Data RAM

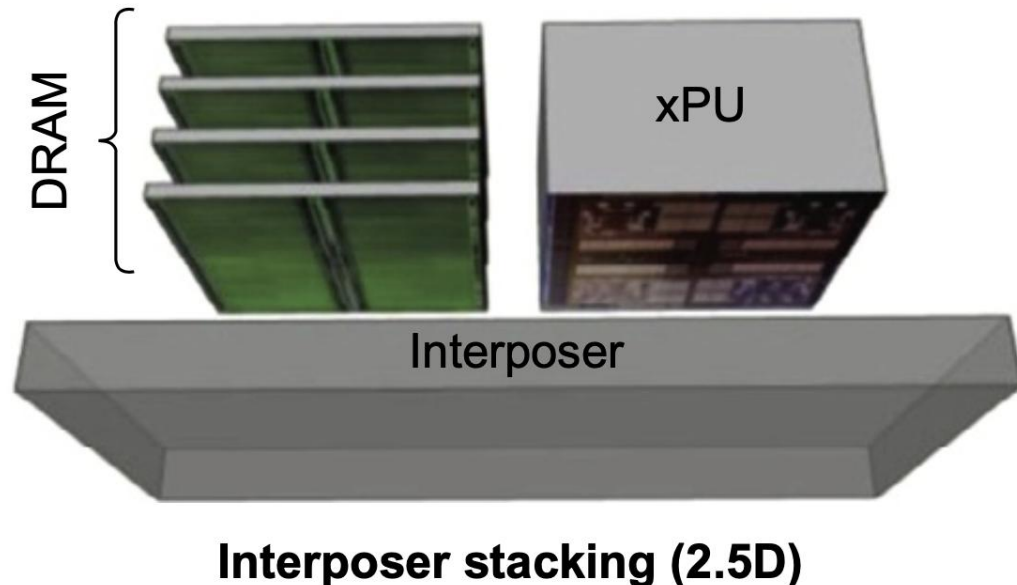
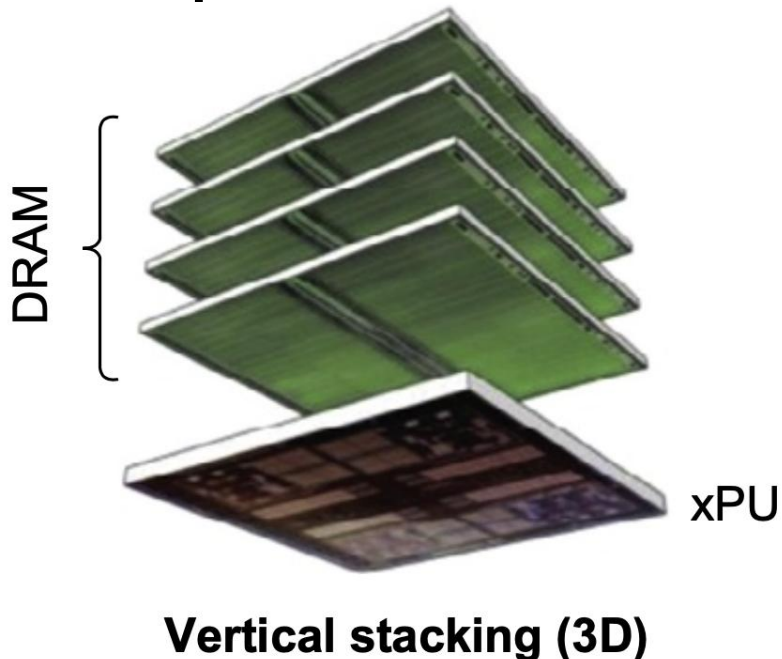
- Wider interfaces
32 bits
- Higher maximum clock rate on data pins
GDRAMs directly connect to GPU
and attach to the board
- 2~5 times the bandwidth per DRAM
versus DDR3 DRAMs

Stacked / Embedded DRAM

- HBM: High Bandwidth Memory
- Packaging innovation, rather than circuit innovation
- Place multiple DRAMs in a stacked or adjacent fashion embedded within the same package as the processor

Stacked / Embedded DRAM

- Lower access latency
- Increase bandwidth by allowing more and faster connections between processor and DRAM



Flash Memory

- A type of **EEPROM** (electronically erasable programmable read-only memory)
- Read-only but can be erased
- Nonvolatile: hold contents w/o any power
- Used as main memory for PMD

Flash Memory

Differences from DRAM

- Must be erased (in blocks) before it is overwritten
- Nonvolatile and less power consumption
- Has a limited number of write cycles for any block
- Cheaper than SDRAM but more expensive than disk
- Slower than SDRAM but faster than disk

Flash Memory

Differences from DRAM

- Has a limited number of write cycles for any block
- **Write leveling technique:** ensure uniform distribution of written blocks throughout the memory

Phase-Change Memory

- Eliminate the need to erase a page before writing
- In comparison with NAND
 - 10 times higher write performance
 - 2~3 times lower read latency
- In comparison with Flash
 - 2~3 times lower read latency
- Suitable for SSD (Solid State Disk)

Memory Dependability

- **Soft errors / Permanent faults**
changes to a cell's contents, not a change in the circuitry
- **Hard errors / Transient faults**
permanent changes in the operation of one or more memory cells

Memory Dependability

Error detection and fix

- **Parity only**

only one bit of overhead to detect a single error in a sequence of bits;

e.g., one parity bit per 8 data bits

- **ECC only**

detect two errors and correct a single error with 8-bit overhead per 64 data bits

Memory Dependability

Error detection and fix

- **Parity only**

only one bit of overhead to detect a single error in a sequence of bits;

e.g., one parity bit per 8 data bits

- **ECC only**

detect two errors and correct a single error with 8-bit overhead per 64 data bits

data loss if failure of single mem chip

Memory Dependability

Error detection and fix

- **Chipkill**

similar to RAID, distribute data and ECC across multiple memory chips;

handle multiple errors and complete failure of a single memory chip;

Memory Dependability

10000-processor server w/ 4 GiB/processor:

Rates of unrecoverable errors in 3 yrs

- **Parity only**

about 90,000, or one unrecoverable (undetected) failure every 17 mins

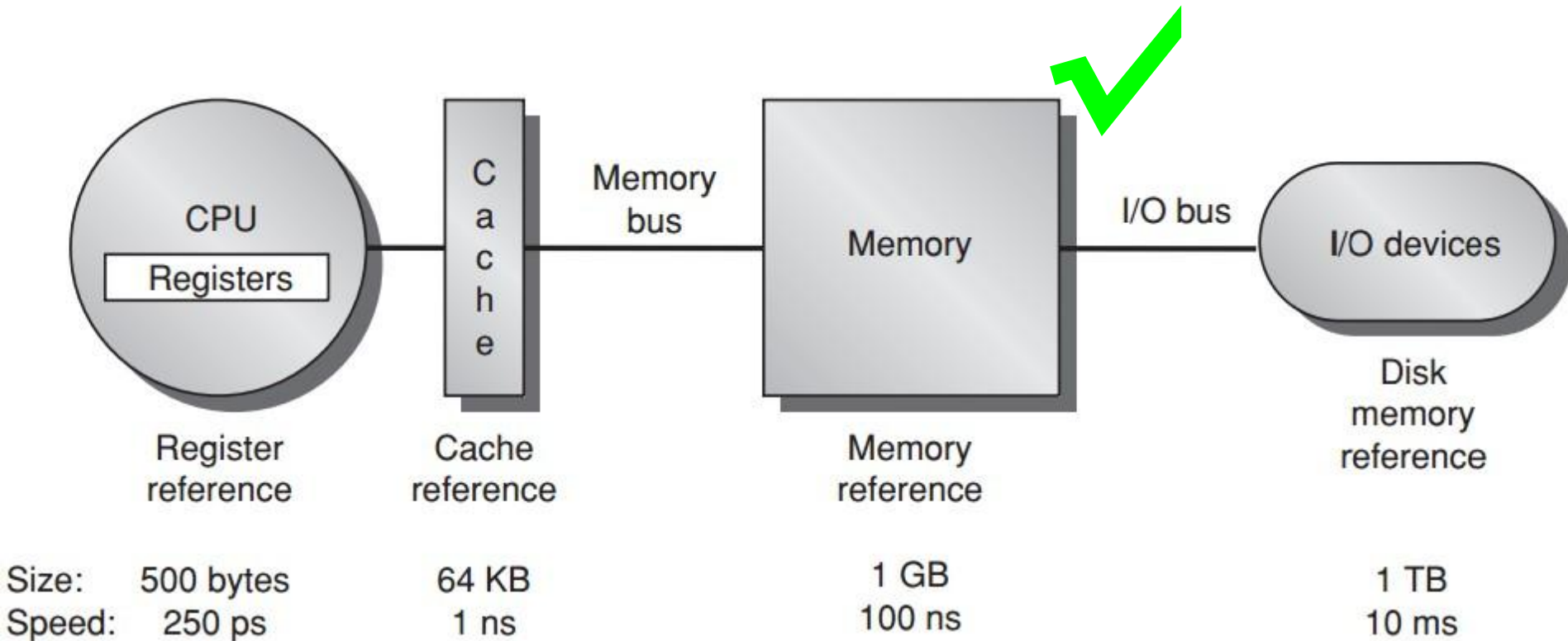
- **ECC only**

about 3,500 or about one undetected or unrecoverable failure every 7.5 hrs

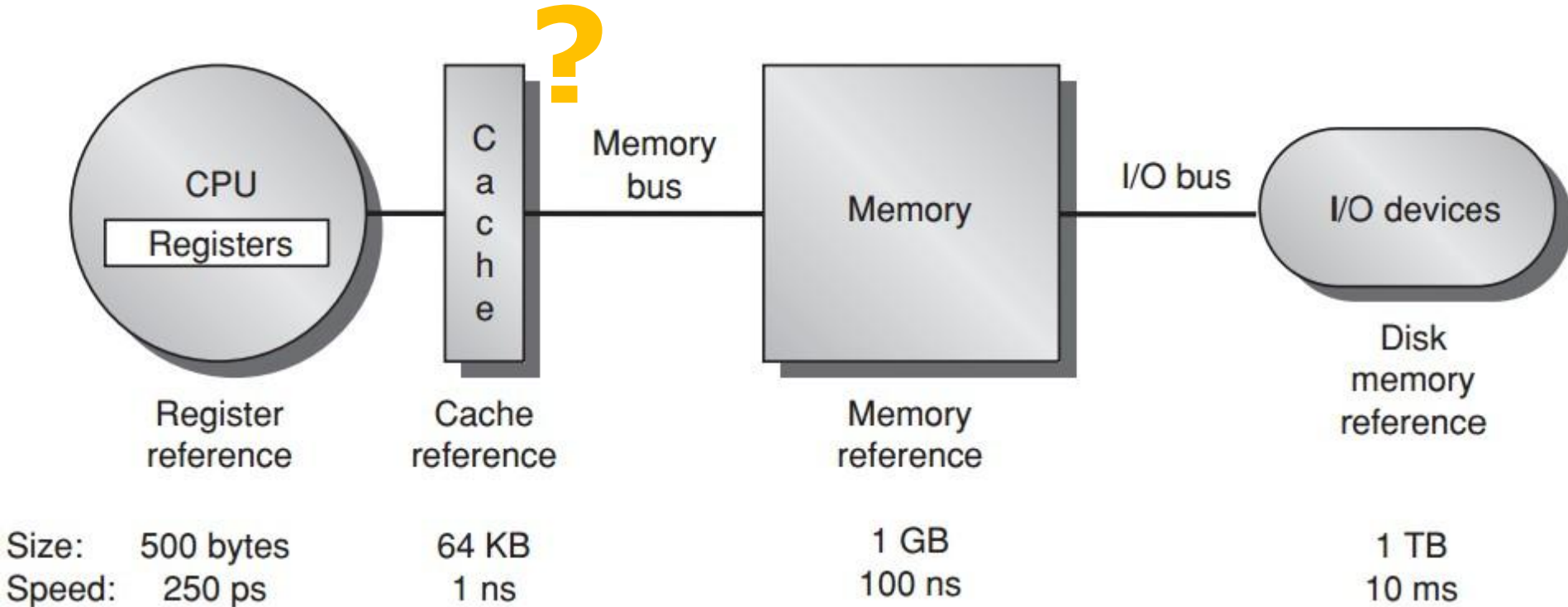
- **Chipkill**

about one undetected or unrecoverable failure every 2 months

Optimization



Optimization



Cache Performance

$$\text{Average Memory Access Time} = (1 - \text{Miss Rate}) \times \text{Hit Time} + \text{Miss Rate} \times \text{Miss Time}$$

Cache Performance

Average Memory Access Time =

$(1 - \text{Miss Rate}) \times \text{Hit Time} + \text{Miss Rate} \times (\text{Hit Time} + \text{Miss Penalty})$

Cache Performance

**Average Memory Access Time =
Hit Time + Miss Rate x Miss Penalty**

six basic optimizations

Six Basic Cache Optimizations

- **1. larger block size**

reduce miss rate; --- *spatial locality*;

reduce static power; --- *lower tag #*;

increase miss penalty, capacity/conflict misses;

- **2. bigger caches**

reduce miss rate; --- *capacity misses*

increase hit time;

increase cost and (static & dynamic) power;

- **3. higher associativity**

reduce miss rate; --- *of conflict misses*;

increase hit time;

increase power;

Six Basic Cache Optimizations

- **4. multilevel caches**

- reduce miss penalty;

- reduce power;

- average memory access time =

- Hit time_{L1} + Miss rate_{L1} x

- (Hit time_{L2} + Miss rate_{L2} x Miss penalty_{L2})

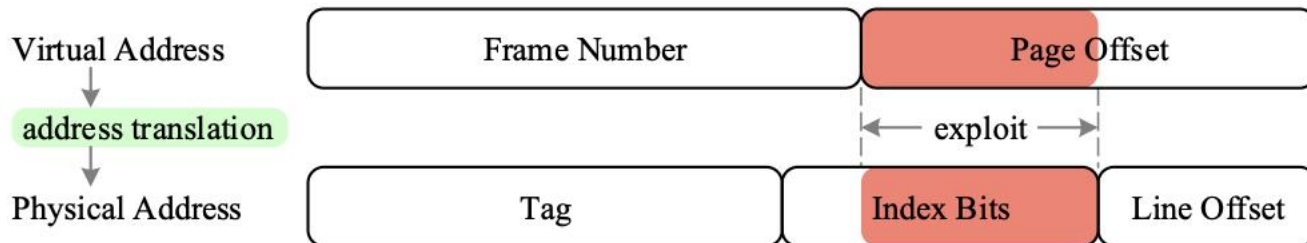
- **5. giving priority to read misses over writes**

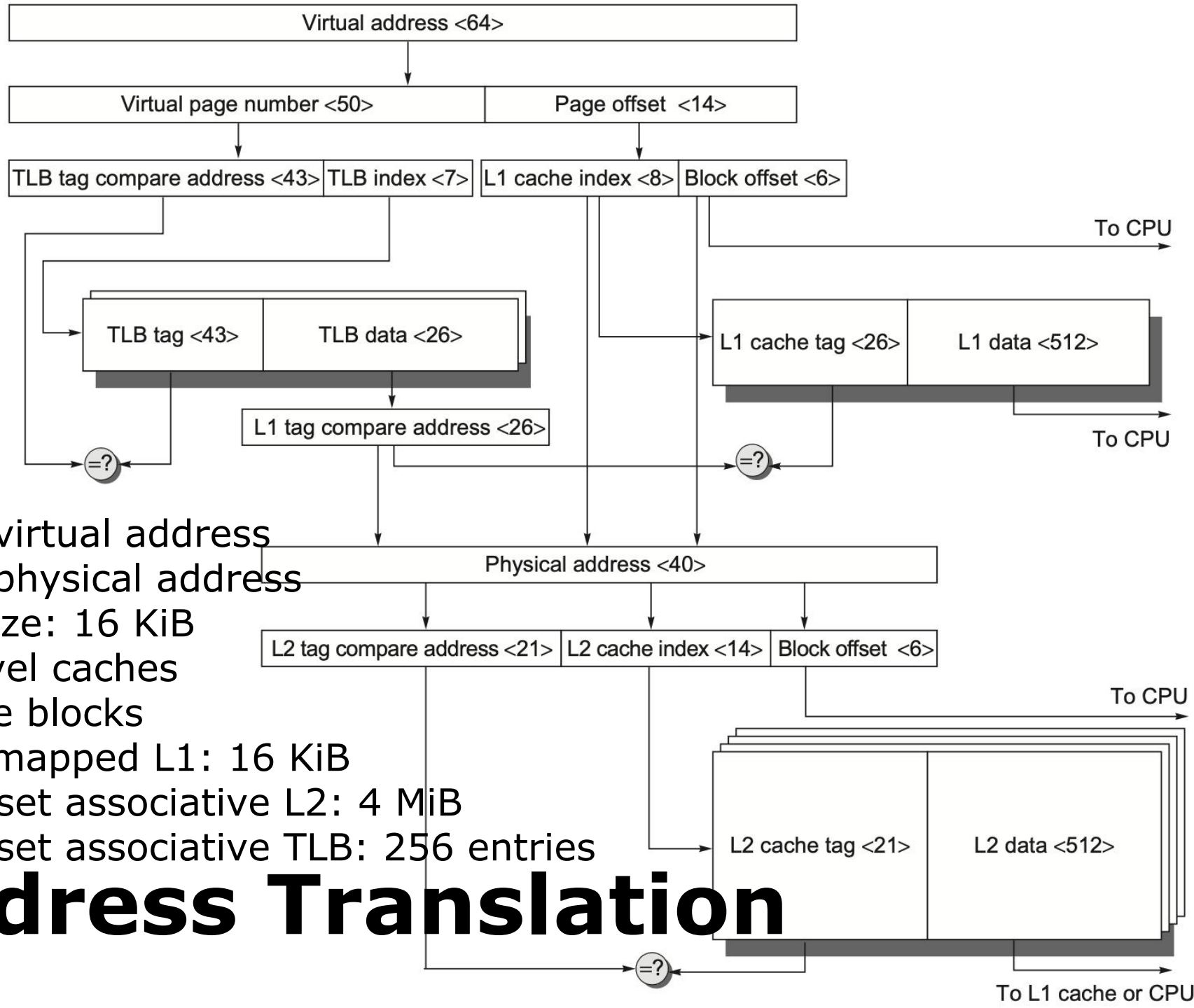
- reduce miss penalty;

- introduce write buffer;

Six Basic Cache Optimizations

- **6. avoiding address translation during indexing of the cache**
reduce hit time;
use page offset to index cache;
virtually indexed, physically tagged;
(indexing cache while querying TLB)





64-bit virtual address
40-bit physical address
page size: 16 KiB
two-level caches
64-byte blocks
direct-mapped L1: 16 KiB
4-way set associative L2: 4 MiB
2-way set associative TLB: 256 entries

Address Translation

Average Memory Access Time ↓ =
Hit Time + Miss Rate x Miss Penalty

Larger block size;

Larger cache size;

Higher associativity;

Root Causes of Miss Rates

- **Compulsory:**

cold-start/first-reference misses;

- **Capacity**

cache size limit;

blocks discarded and later retrieved;

- **Conflict**

collision misses: associativity

direct mapped or set associative cache;

a block discarded and later retrieved in a set;

Capacity vs Conflict

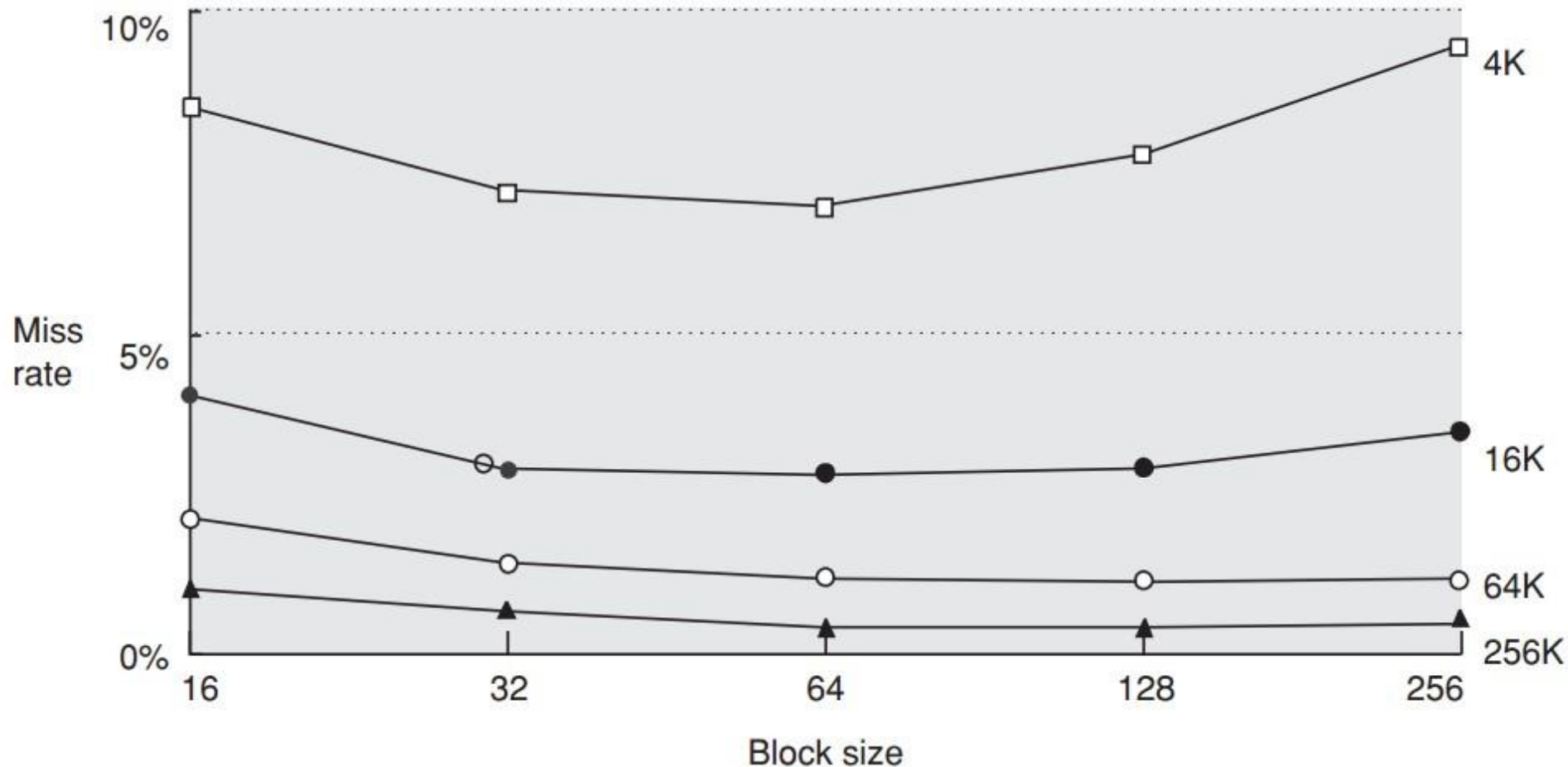
- Capacity misses occur upon the cache is full
- Not necessarily for conflict misses to take place
- Conflict misses can occur without the cache being fully occupied

Opt #1: Larger Block Size

- **Reduce** compulsory misses
- Leverage spatial locality

- **Increase** conflict/capacity misses
- Fewer blocks in the cache
- Larger blocks increase miss penalty

Opt #1: Larger Block Size



Opt #1: Larger Block Size

Block size	Cache size			
	4K	16K	64K	256K
16	8.57%	3.94%	2.04%	1.09%
32	7.24%	2.87%	1.35%	0.70%
64	7.00%	2.64%	1.06%	0.51%
128	7.78%	2.77%	1.02%	0.49%
256	9.51%	3.29%	1.15%	0.49%

given the above miss rates;
assume memory takes 80 CC overhead,
delivers 16 bytes in 2 CC;

Q: which block size has the smallest average
memory access time for each cache size?

Opt #1: Larger Block Size

256 byte

9.51%

3.29%

1.15%

• Answer

avg mem access time

= hit time + miss rate x miss penalty

*assume 1-CC hit time

for a 256-byte block in a 256 KB cache:

avg mem access time

= $1 + \underline{0.49\%} \times [80 + 2 \times (256/16)]$

= 1.5 cc

Opt #1: Larger Block Size

- **Answer**

average memory access time

Block size	Miss penalty	Cache size			
		4K	16K	64K	256K
16	82	8.027	4.231	2.673	1.894
32	84	7.082	3.411	2.134	1.588
64	88	7.160	3.323	1.933	1.449
128	96	8.469	3.659	1.979	1.470
256	112	11.651	4.685	2.288	1.549

Opt #2: Larger Cache

- **Reduce** capacity misses
- **Increase** hit time, cost, and power

Opt #3: Higher Associativity

- **Reduce** conflict misses
- **Increase** hit time

Opt #3: Higher Associativity

- **Reduce** conflict misses
- **Increase** hit time
- 2:1 cache rule of thumb:
a direct-mapped cache of size N
has about the same miss rate
as a two-way set associative cache of
size $N/2$

Opt #3: Higher Associativity

- **Example**

assume higher associativity -> higher clock cycle time:

$$\text{Clock cycle time}_{2\text{-way}} = 1.36 \times \text{Clock cycle time}_{1\text{-way}}$$

$$\text{Clock cycle time}_{4\text{-way}} = 1.44 \times \text{Clock cycle time}_{1\text{-way}}$$

$$\text{Clock cycle time}_{8\text{-way}} = 1.52 \times \text{Clock cycle time}_{1\text{-way}}$$

assume 1-cc hit time,

25-cc miss penalty,

and miss rates in the following table:

• Miss rates

Cache size (KB)	Degree associative	Total miss rate	Miss rate components (relative percent) (sum = 100% of total miss rate)					
			Compulsory		Capacity		Conflict	
4	1-way	0.098	0.0001	0.1%	0.070	72%	0.027	28%
4	2-way	0.076	0.0001	0.1%	0.070	93%	0.005	7%
4	4-way	0.071	0.0001	0.1%	0.070	99%	0.001	1%
4	8-way	0.071	0.0001	0.1%	0.070	100%	0.000	0%
8	1-way	0.068	0.0001	0.1%	0.044	65%	0.024	35%
8	2-way	0.049	0.0001	0.1%	0.044	90%	0.005	10%
8	4-way	0.044	0.0001	0.1%	0.044	99%	0.000	1%
8	8-way	0.044	0.0001	0.1%	0.044	100%	0.000	0%
16	1-way	0.049	0.0001	0.1%	0.040	82%	0.009	17%
16	2-way	0.041	0.0001	0.2%	0.040	98%	0.001	2%
16	4-way	0.041	0.0001	0.2%	0.040	99%	0.000	0%
16	8-way	0.041	0.0001	0.2%	0.040	100%	0.000	0%
32	1-way	0.042	0.0001	0.2%	0.037	89%	0.005	11%
32	2-way	0.038	0.0001	0.2%	0.037	99%	0.000	0%
32	4-way	0.037	0.0001	0.2%	0.037	100%	0.000	0%
32	8-way	0.037	0.0001	0.2%	0.037	100%	0.000	0%
64	1-way	0.037	0.0001	0.2%	0.028	77%	0.008	23%
64	2-way	0.031	0.0001	0.2%	0.028	91%	0.003	9%
64	4-way	0.030	0.0001	0.2%	0.028	95%	0.001	4%
64	8-way	0.029	0.0001	0.2%	0.028	97%	0.001	2%
128	1-way	0.021	0.0001	0.3%	0.019	91%	0.002	8%
128	2-way	0.019	0.0001	0.3%	0.019	100%	0.000	0%
128	4-way	0.019	0.0001	0.3%	0.019	100%	0.000	0%
128	8-way	0.019	0.0001	0.3%	0.019	100%	0.000	0%
256	1-way	0.013	0.0001	0.5%	0.012	94%	0.001	6%
256	2-way	0.012	0.0001	0.5%	0.012	99%	0.000	0%
256	4-way	0.012	0.0001	0.5%	0.012	99%	0.000	0%
256	8-way	0.012	0.0001	0.5%	0.012	99%	0.000	0%
512	1-way	0.008	0.0001	0.8%	0.005	66%	0.003	33%
512	2-way	0.007	0.0001	0.9%	0.005	71%	0.002	28%
512	4-way	0.006	0.0001	1.1%	0.005	91%	0.000	8%
512	8-way	0.006	0.0001	1.1%	0.005	95%	0.000	4%

Opt #3: Higher Associativity

- **Question:**

for which cache sizes

are each of the statements true?

Average memory access time_{8-way} < Average memory access time_{4-way}

Average memory access time_{4-way} < Average memory access time_{2-way}

Average memory access time_{2-way} < Average memory access time_{1-way}

Opt #3: Higher Associativity

- **Answer**

example, for a 512 KB, 8-way set associative cache:

avg mem access time

= hit time + miss rate x miss penalty

= $1.52 \times 1 + 0.006 \times 25$

= 1.66

Opt #3: Higher Associativity

- **Answer**

average memory access time

Cache size (KB)	Associativity			
	One-way	Two-way	Four-way	Eight-way
4	3.44	3.25	3.22	3.28
8	2.69	2.58	2.55	2.62
16	2.23	2.40	2.46	2.53
32	2.06	2.30	2.37	2.45
64	1.92	2.14	2.18	2.25
128	1.52	1.84	1.92	2.00
256	1.32	1.66	1.74	1.82
512	1.20	1.55	1.59	1.66

Average Memory Access Time ↓ =
Hit Time + Miss Rate x Miss Penalty

Multilevel caches;

Reads > Writes;

Opt #4: Multilevel Cache

- **Reduce** miss penalty

- **Motivation**

faster/smaller cache to keep pace with the speed of processors?

larger cache to overcome the widening gap between processor and main mem?

Opt #4: Multilevel Cache

- **Reduce** miss penalty

- **Motivation**

faster/smaller cache to keep pace with the speed of processors?

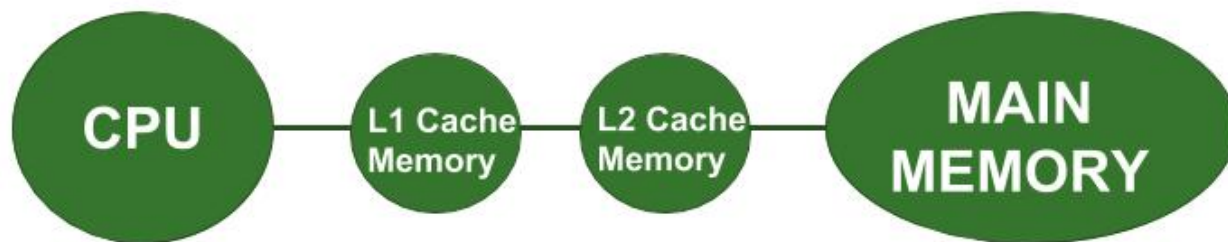
larger cache to overcome the widening gap between processor and main mem?

do both

Opt #4: Multilevel Cache

- **Two-level cache**

Add another level of cache between the original cache and memory

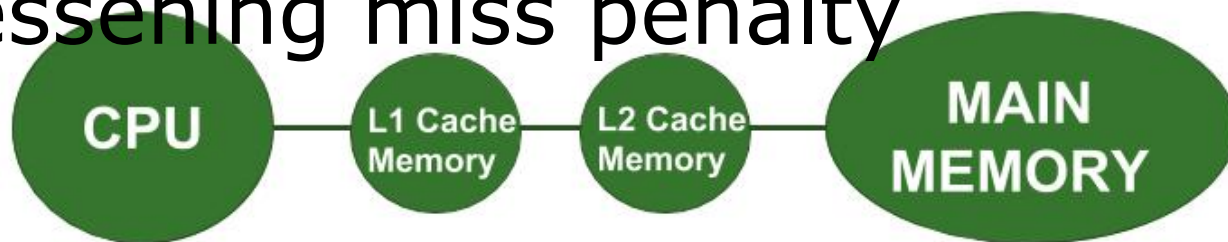


Opt #4: Multilevel Cache

- **Two-level cache**

Add another level of cache between the original cache and memory

- **L1**: small enough to match the clock cycle time of the fast processor;
- **L2**: large enough to capture many accesses that would go to main memory, lessening miss penalty



Opt #4: Multilevel Cache

- **Average memory access time**

$$\begin{aligned} &= \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times \text{Miss penalty}_{L1} \\ &= \text{Hit time}_{L1} + \text{Miss rate}_{L1} \\ &\quad \times (\text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2}) \end{aligned}$$

- **Average mem stalls per instruction**

$$\begin{aligned} &= \text{Misses per instruction}_{L1} \times \text{Hit time}_{L2} \\ &\quad + \text{Misses per instr}_{L2} \times \text{Miss penalty}_{L2} \end{aligned}$$

Opt #4: Multilevel Cache

- **Average memory access time**

$$\begin{aligned} &= \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times \text{Miss penalty}_{L1} \\ &= \text{Hit time}_{L1} + \text{Miss rate}_{L1} \\ &\quad \times (\text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2}) \end{aligned}$$

- **Average mem stalls per instruction**

$$\begin{aligned} &= \text{Misses per instruction}_{L1} \times \text{Hit time}_{L2} \\ &\quad + \text{Misses per instr}_{L2} \times \text{Miss penalty}_{L2} \end{aligned}$$

(recap) Cache Performance: Equations!!!

$$\text{CPU execution time} = (\text{CPU clock cycles} + \text{Memory stall cycles}) \times \text{Clock cycle time}$$

Assumption:
Includes the time
to handle
a cache hit/miss

$$\text{Memory stall cycles} = \text{Number of misses} \times \text{Miss penalty}$$

$$= IC \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}$$

$$= IC \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss rate} \times \text{Miss penalty}$$

$$\begin{aligned} \text{Memory stall clock cycles} = & IC \times \text{Reads per instruction} \times \text{Read miss rate} \times \text{Read miss penalty} \\ & + IC \times \text{Writes per instruction} \times \text{Write miss rate} \times \text{Write miss penalty} \end{aligned}$$

Opt #4: Multilevel Cache

- **Local miss rate**

the number of misses in a cache
divided by the total number of mem
accesses to this cache;

Miss rate_{L1}, Miss rate_{L2}

- **Global miss rate**

the number of misses in the cache
divided by the number of mem
accesses generated **by the processor**;

Miss rate_{L1}, Miss rate_{L1} x Miss rate_{L2}

Opt #4: Multilevel Cache

- **Local miss rate**

the number of misses in a cache
divided by the total number of mem
accesses to this cache;

Miss rate_{L1}, Miss rate_{L2}

$$\begin{aligned} & L1 \times \text{Mrate}_{L2} / \text{all} \\ & = (L1 / \text{all}) \times \text{Mrate}_{L2} \\ & = \text{Mrate}_{L1} \times \text{Mrate}_{L2} \end{aligned}$$

- **Global miss rate**

the number of misses in the cache
divided by the number of mem
accesses generated by the processor;

Miss rate_{L1}, Miss rate_{L1} x Miss rate_{L2}

Opt #4: Multilevel Cache

- **Multilevel inclusion**

L1 data always present in L2;
ease consistency checking by targeting only L2 cache;

- **Multilevel exclusion**

for L2 slightly bigger than L1,
L1 data are never found in L2;
L1 miss results in a swap of blocks
between L1 and L2;

Opt #4: Multilevel Cache

- **Example**

1000 mem references -> 40 misses in L1 and 20 misses in L2;

miss penalty from L2 is 200 cc;

hit time of L2 is 10 cc;

hit time of L1 is 1 cc;

1.5 mem references per instruction;

Q: 1. various miss rates?

2. avg mem access time?

3. avg stall cycles per instruction?

Opt #4: Multilevel Cache

- **Answer**

1. various miss rates?

L1: local = global

$$40/1000 = 4\%$$

L2:

$$\text{local: } 20/40 = 50\%$$

$$\text{global: } 20/1000 = 2\%$$

Opt #4: Multilevel Cache

- **Answer**

2. avg mem access time?

average memory access time

$$\begin{aligned} &= \text{Hit time}_{L1} + \text{Miss rate}_{L1} \\ &\quad \times (\text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2}) \\ &= 1 + 4\% \times (10 + 50\% \times 200) \\ &= 5.4 \end{aligned}$$

Opt #4: Multilevel Cache

- **Answer**

3. avg stall cycles per instruction?

average stall cycles per instruction

$$\begin{aligned} &= \text{Misses per instruction}_{L1} \times \text{Hit time}_{L2} \\ &\quad + \text{Misses per instr}_{L2} \times \text{Miss penalty}_{L2} \\ &= (1.5 \times 40 / 1000) \times 10 + (1.5 \times 20 / 1000) \times 200 \\ &= 6.6 \end{aligned}$$

Opt #5: Prioritize Read Misses Over Writes

- **Reduce** miss penalty
- not simply stall read miss until write buffer empties,
check the contents of write buffer,
let the read miss continue
if no conflicts with write buffer &
memory system is available

Opt #5: Prioritize Read Misses Over Writes

- **Why**

```
SW R3, 512(R0)    ;M[512] ← R3      (cache index 0)
LW R1, 1024(R0)   ;R1 ← M[1024]     (cache index 0)
LW R2, 512(R0)    ;R2 ← M[512]     (cache index 0)
```

for the code sequence, assume a direct-mapped, **write-through** cache that maps 512 and 1024 to the same block;

a four-word write buffer is **not** checked on a read miss.

R2.value $\equiv$ R3.value ?

*read-after-write data hazard

Average Memory Access Time ↓ =
Hit Time + Miss Rate x Miss Penalty

Avoid address translation

during indexing of the cache

Opt #6: Avoid Address Translation During Indexing Cache

- **Cache** addressing
 - virtual address – *virtual cache*
 - physical address – *physical cache*
- **Processor/program** – virtual address
- Processor -> address translation -> Cache

virtual cache or physical cache?

Opt #6: Avoid Address Translation During Indexing Cache

- **Virtually indexed, physically tagged**
page offset to index the cache;
physical address for tag match;
- For direct-mapped cache,
it cannot be bigger than the page size.

- *Reference: CPU Cache*

<http://zh.wikipedia.org/wiki/CPU%E9%AB%98%E9%80%9F%E7%BC%93%E5%AD%98>

*more in later lectures

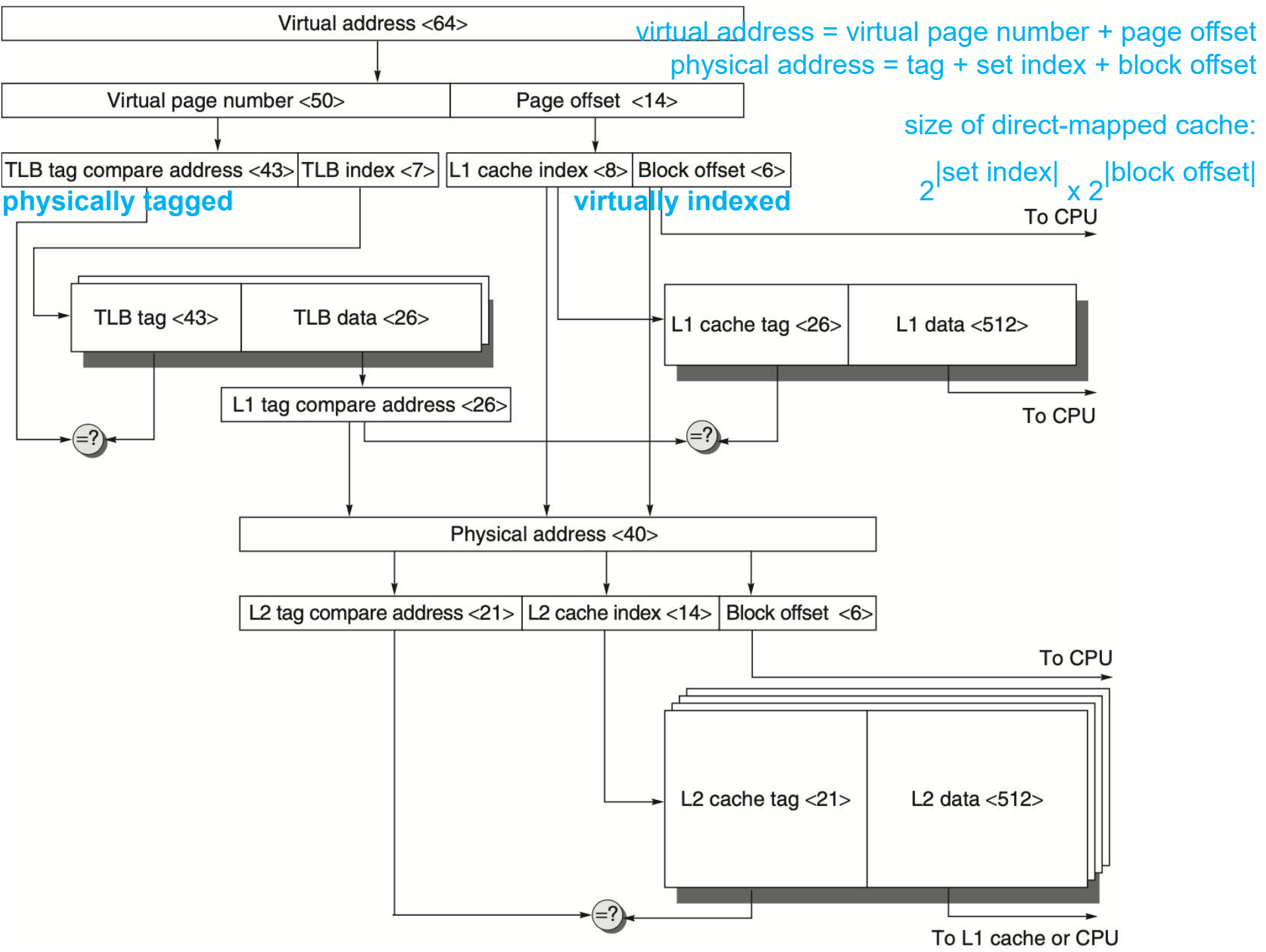
Opt #6: Avoid Address Translation During Indexing Cache

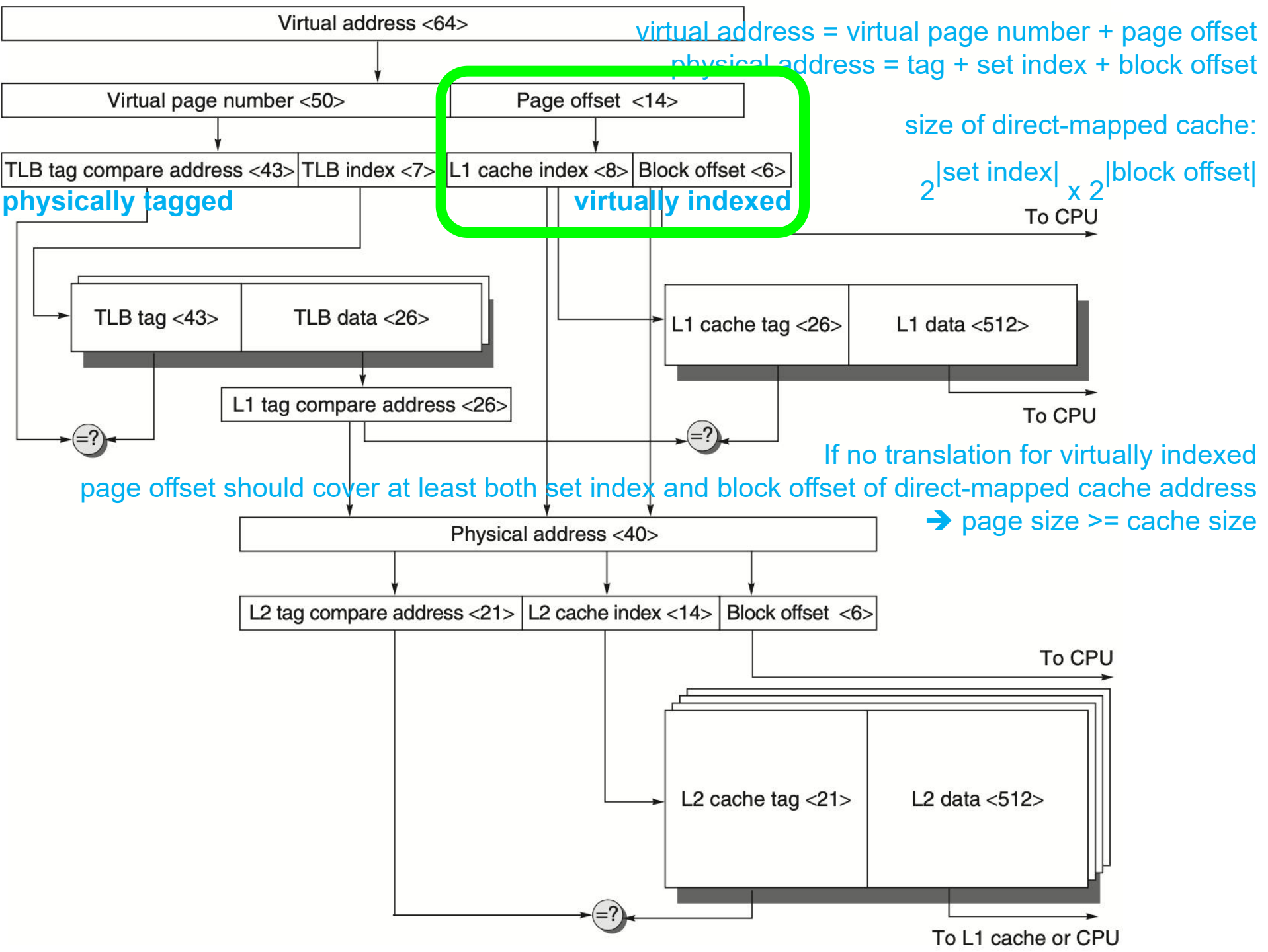
- **Virtually indexed, physically tagged**
page offset to index the cache;
physical address for tag match;
- For direct-mapped cache,
it cannot be bigger than the page size???

- *Reference: CPU Cache*

<http://zh.wikipedia.org/wiki/CPU%E9%AB%98%E9%80%9F%E7%BC%93%E5%AD%98>

*more in later lectures





How virtual memory works?

How virtual memory works?

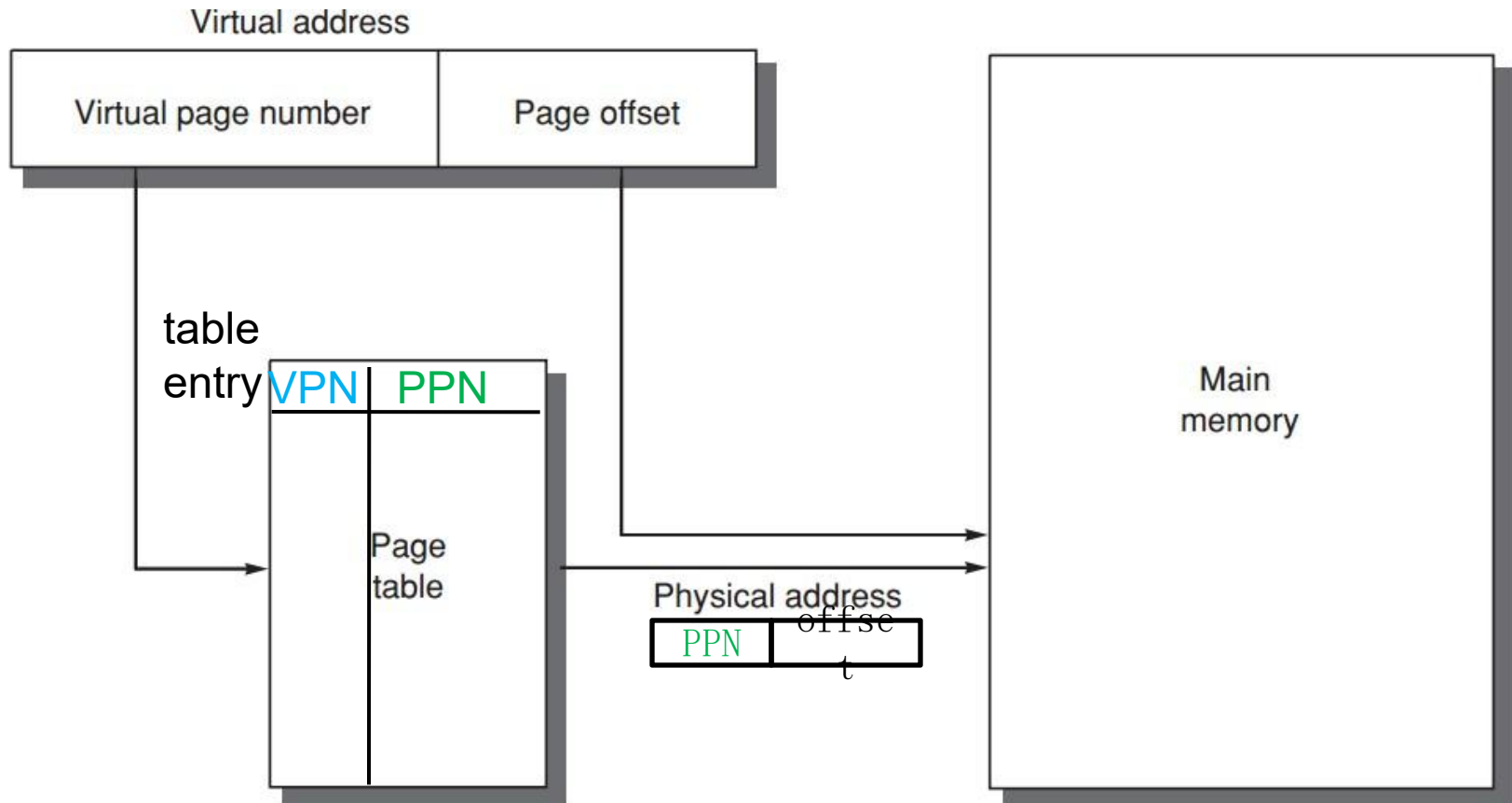
Four Questions

Four Mem Hierarchy Q's

- **Q1. *Where to place a block?***
- **Fully associative** strategy:
OS allows blocks to be placed anywhere in main memory
- Because of high miss penalty by access to a rotating magnetic storage device upon page/address fault

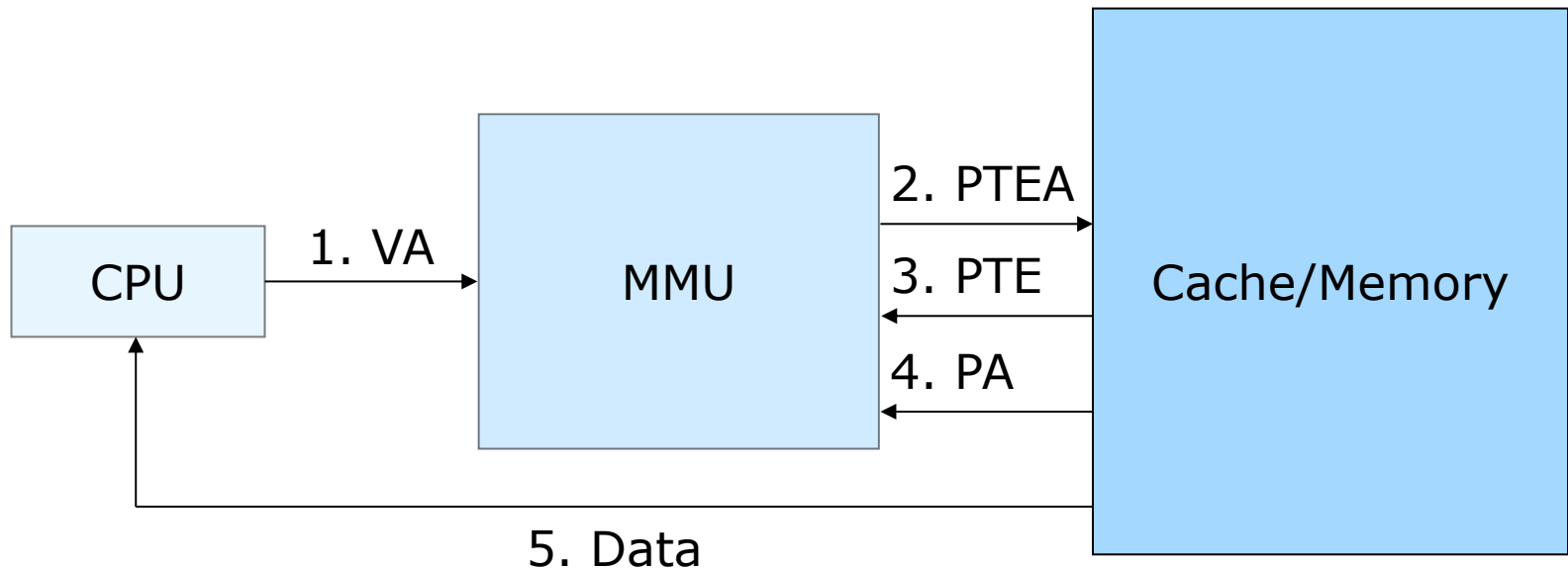
Four Mem Hierarchy Q's

- **Q2. *How to find a block?***



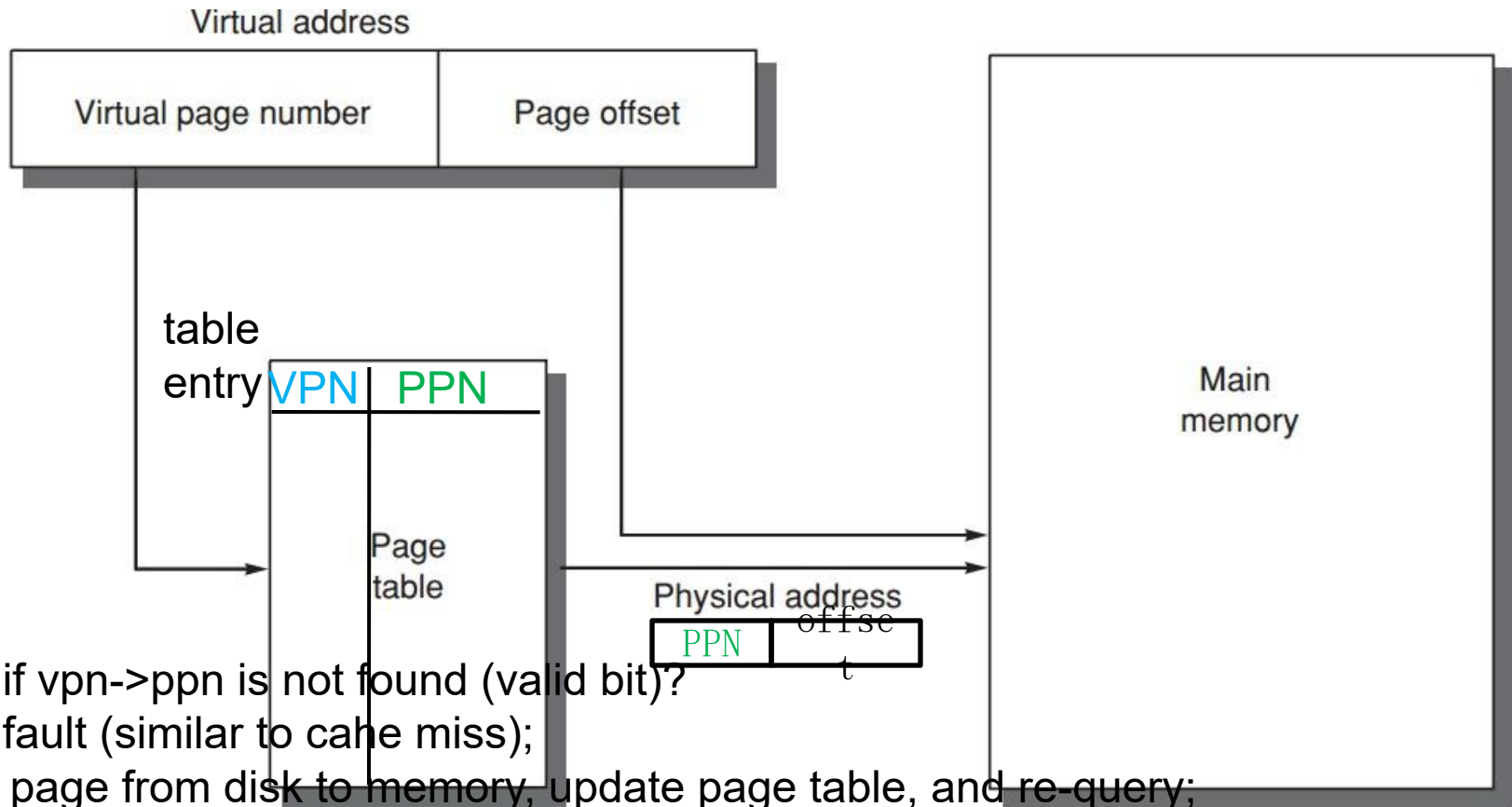
Four Mem Hierarchy Q's

- **Q2. *How to find a block?***
- Access memory twice:
one for accessing PTE address
one for accessing data using phy addr



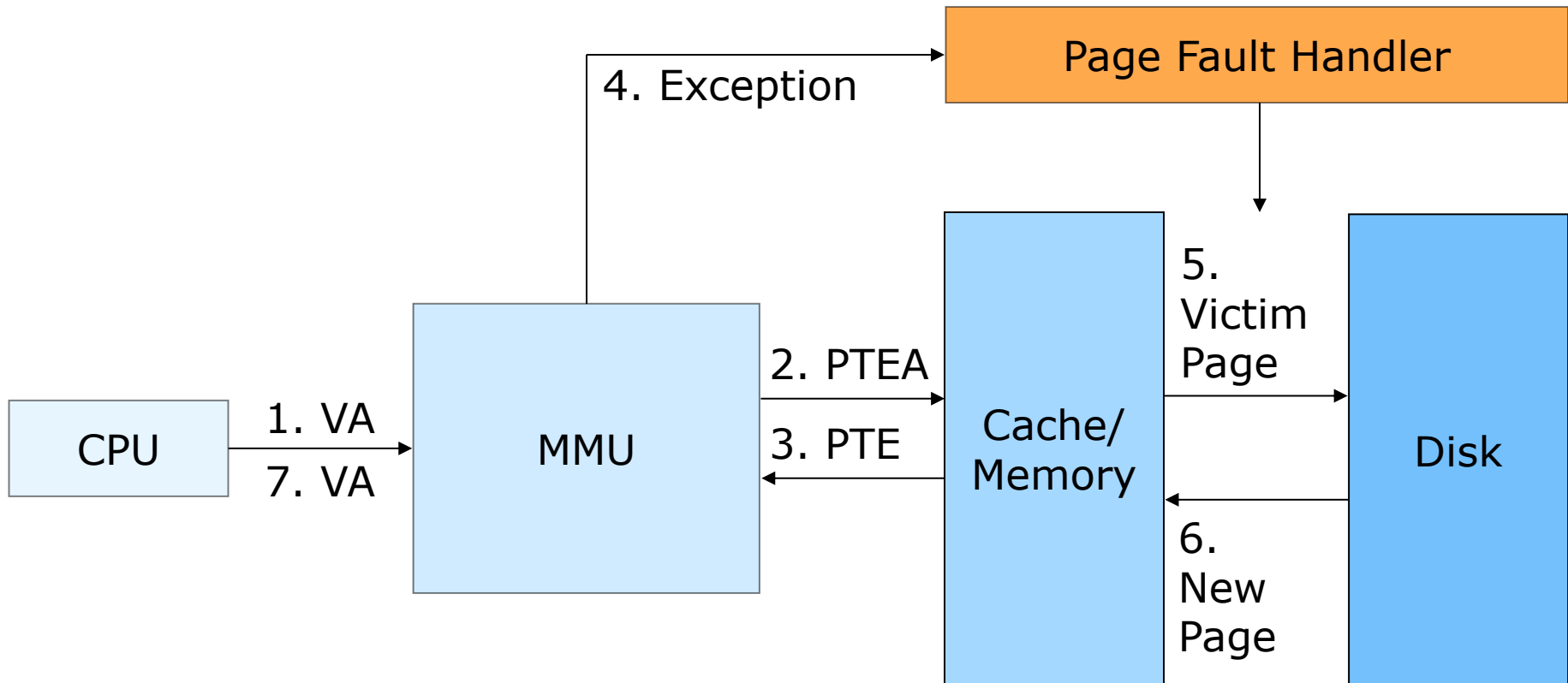
Four Mem Hierarchy Q's

- **Q2. *How to find a block?***



Four Mem Hierarchy Q's

- **Q2. *How to find a block?***
- Processing a page fault



Four Memory Hierarchy Q's

- **Q3. Which block to replace upon a virtual memory miss?**
- Least recently used (LRU) block
- use/reference bit
 - logically set whenever a page is accessed;
 - OS periodically clears use bits and later records them to track the least recently referenced pages;

Four Mem Hierarchy Q's

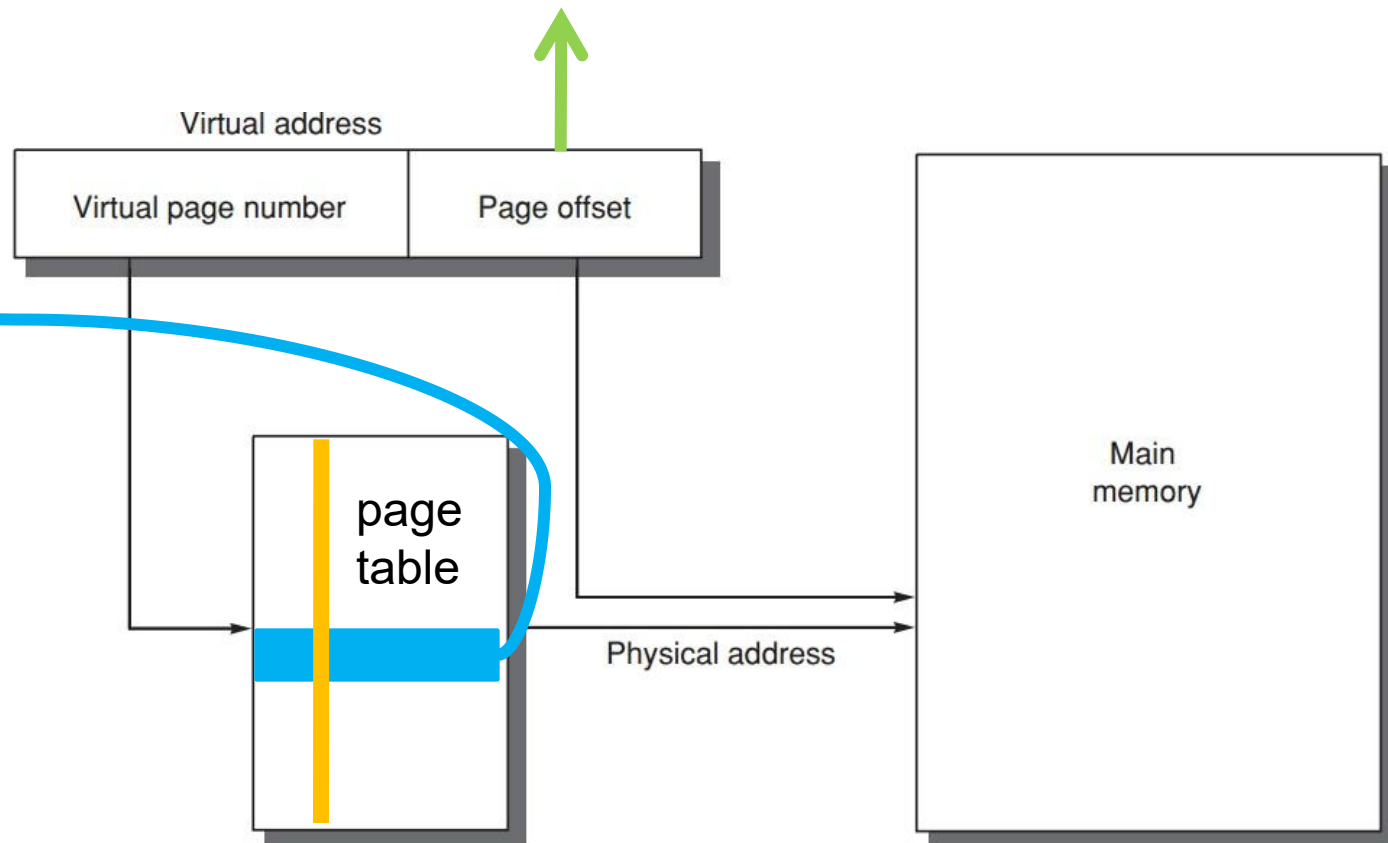
- **Q4. *What happens on a write?***
- Write-back strategy
as accessing rotating magnetic disk
takes millions of clock cycles;
- Dirty bit
write a block to disk only if it has been
altered since being read from the disk;

well...tell me more

well...tell me more
Address Translation

Page Table

physical page number || page offset = physical address



Page Table ?

- Page tables are often large

32-bit virtual address, 4KB pages,
4 bytes per page table entry.

page table size:

$$(2^{32}/2^{12}) \times 2^2 = 2^{22} \text{ bytes} = 4 \text{ MB}$$

Page Table ?

- Page tables are stored in main memory
- Logically two memory accesses for data access:
 - one to obtain the physical address from page table;
 - one to get the data from the physical address;

Access time doubled

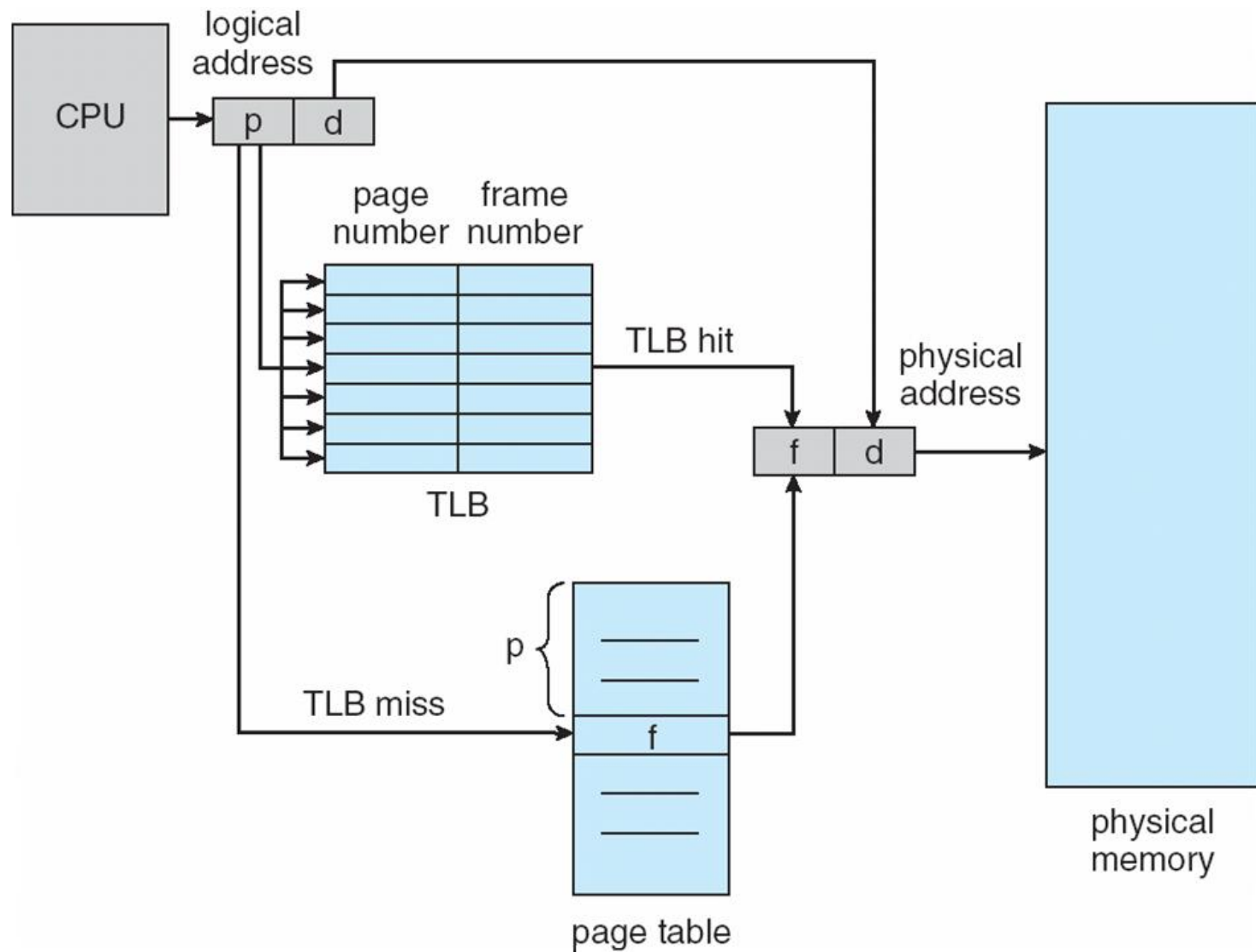
How to be faster?

cache!

Learn from History

- **Translation lookaside buffer (TLB)**
/translation buffer (TB)
a special **cache!**
that keeps (prev) address translations
- TLB entry
 - tag: portions of the virtual address;
 - data: a physical page frame number, protection field, valid bit, use bit, dirty bit;

TLB

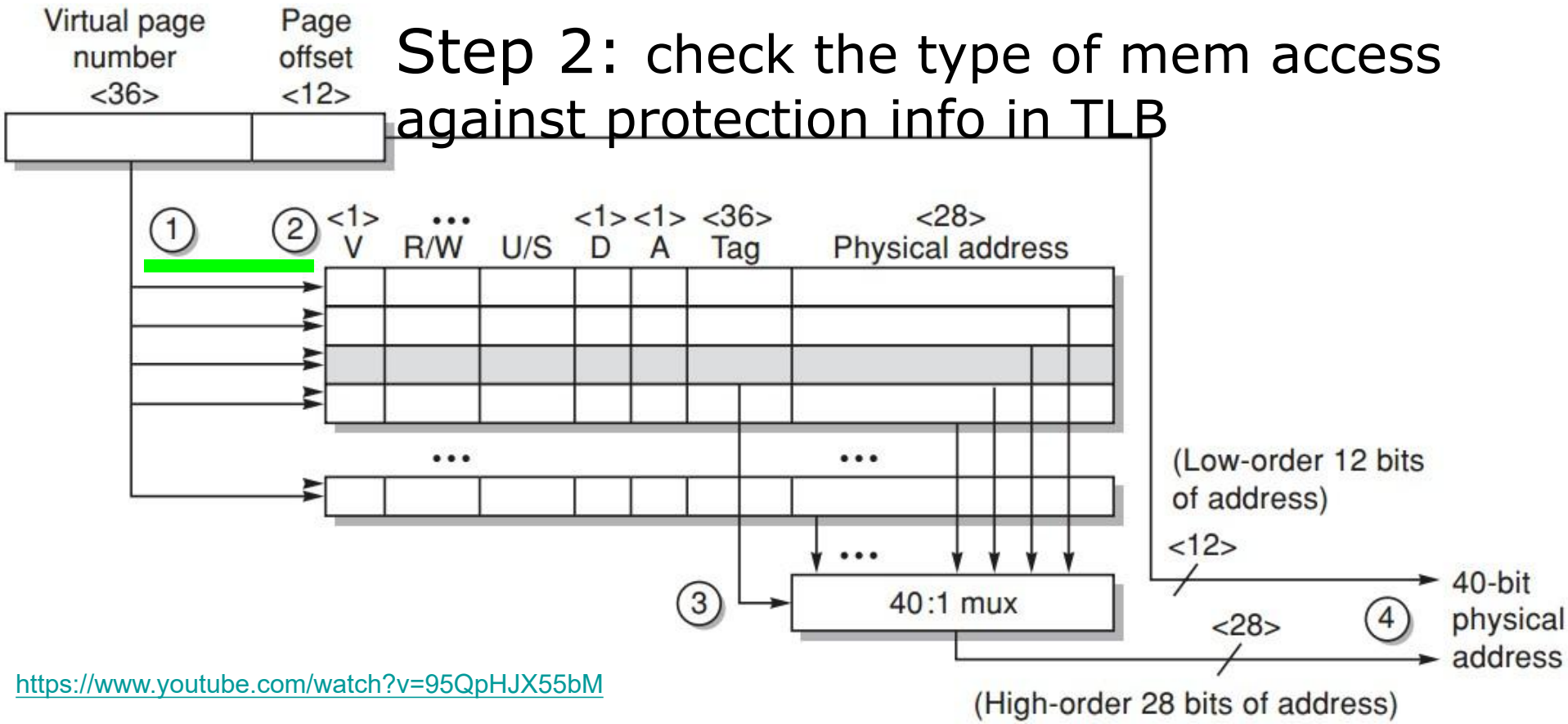


TLB Example

- **Opteron data TLB**

Steps 1&2: send the virtual address to all tags

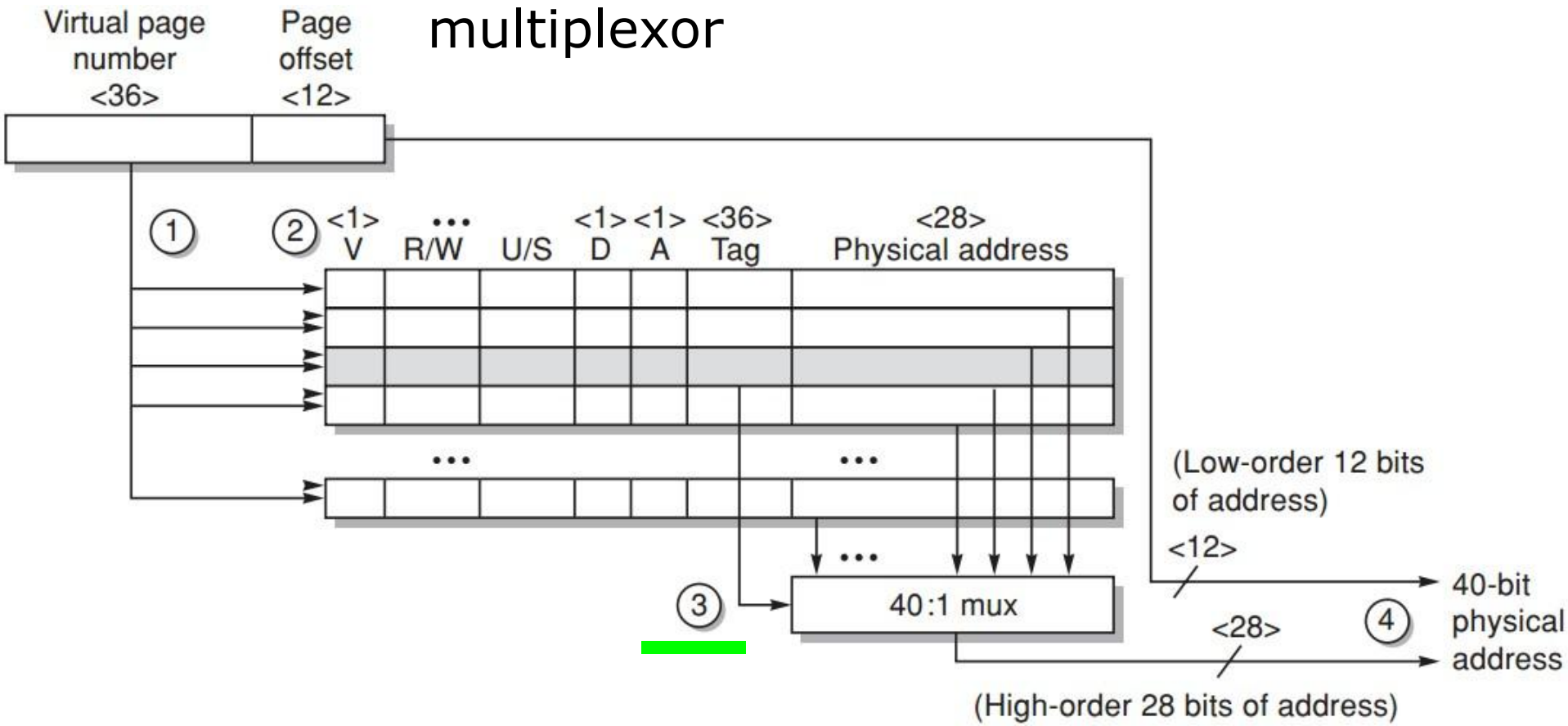
Step 2: check the type of mem access against protection info in TLB



TLB Example

- **Opteron data TLB**

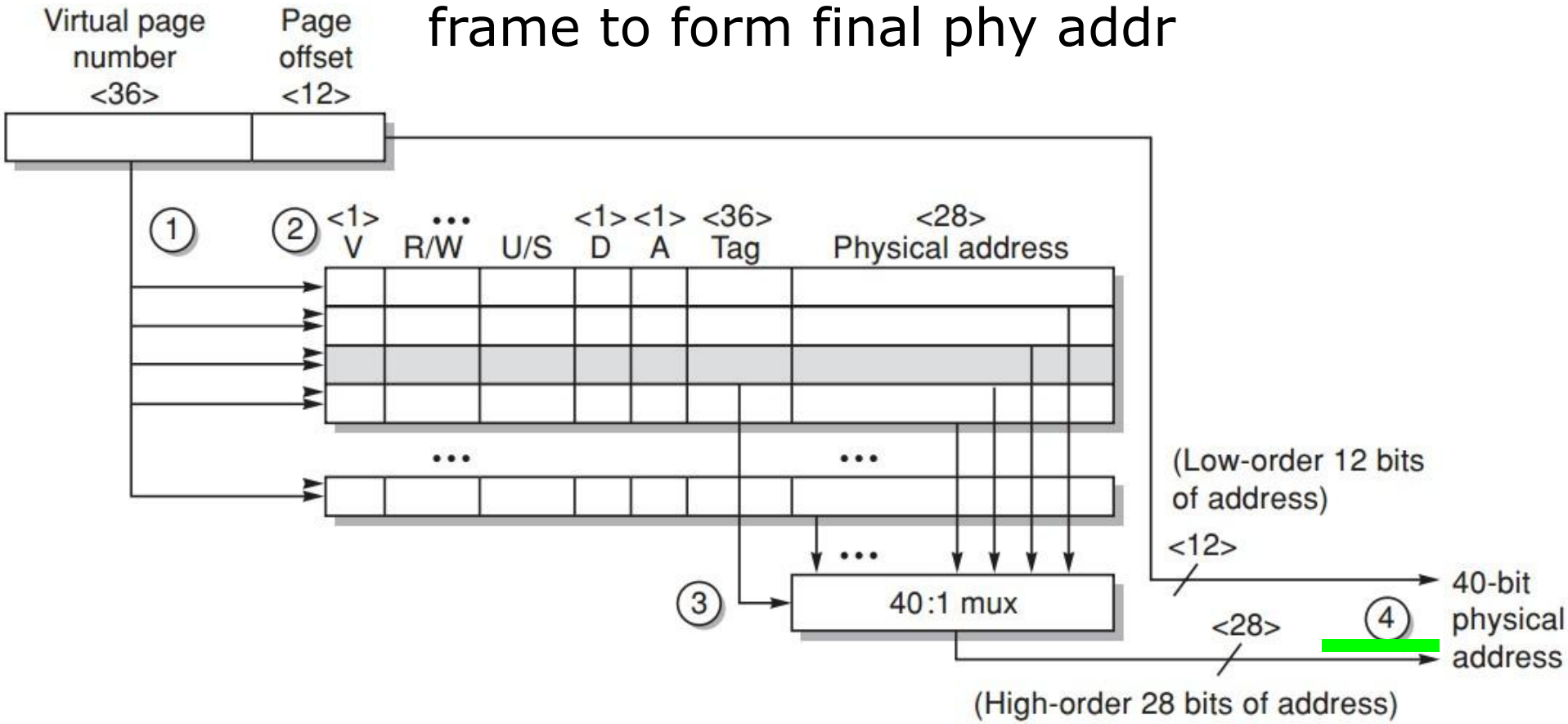
Steps 3: the matching tag sends phy addr through multiplexor



TLB Example

- **Opteron data TLB**

Steps 4: concatenate page offset to phy page frame to form final phy addr



TLB Example

• Opteron data TLB

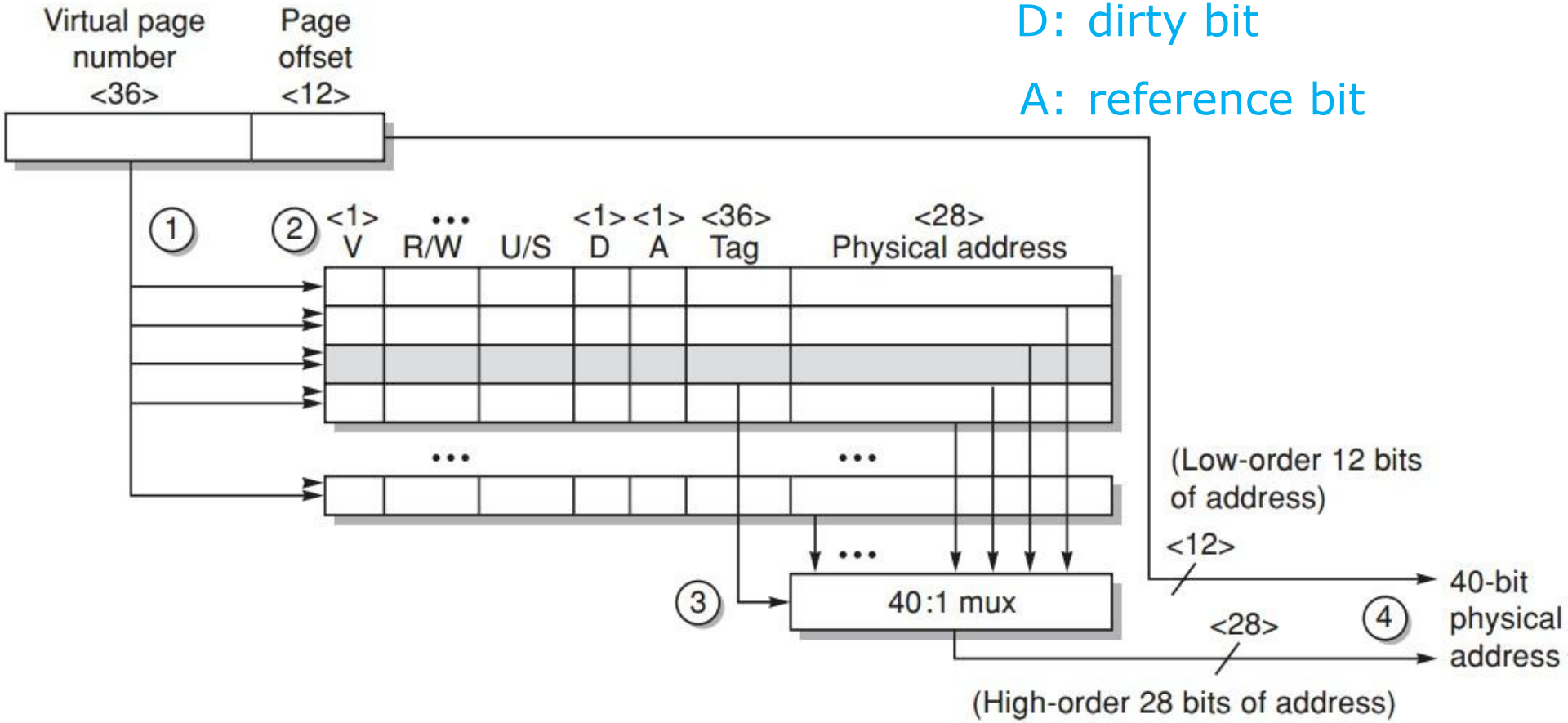
V: valid bit

R/W: read/write permission

U/S: user/supervisor mode

D: dirty bit

A: reference bit



Does page size matter?

Page Size Selection

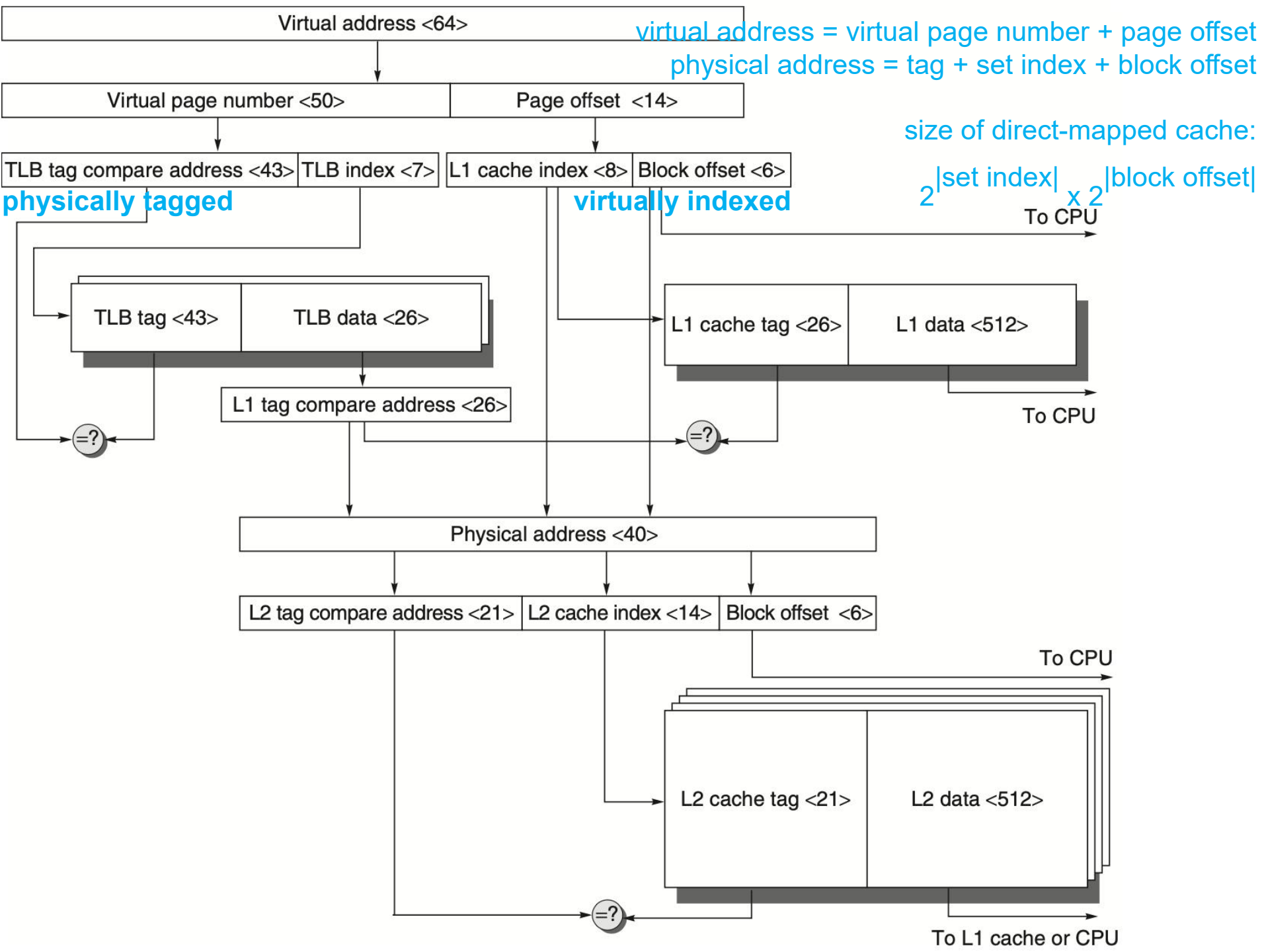
Pros of **larger** page size

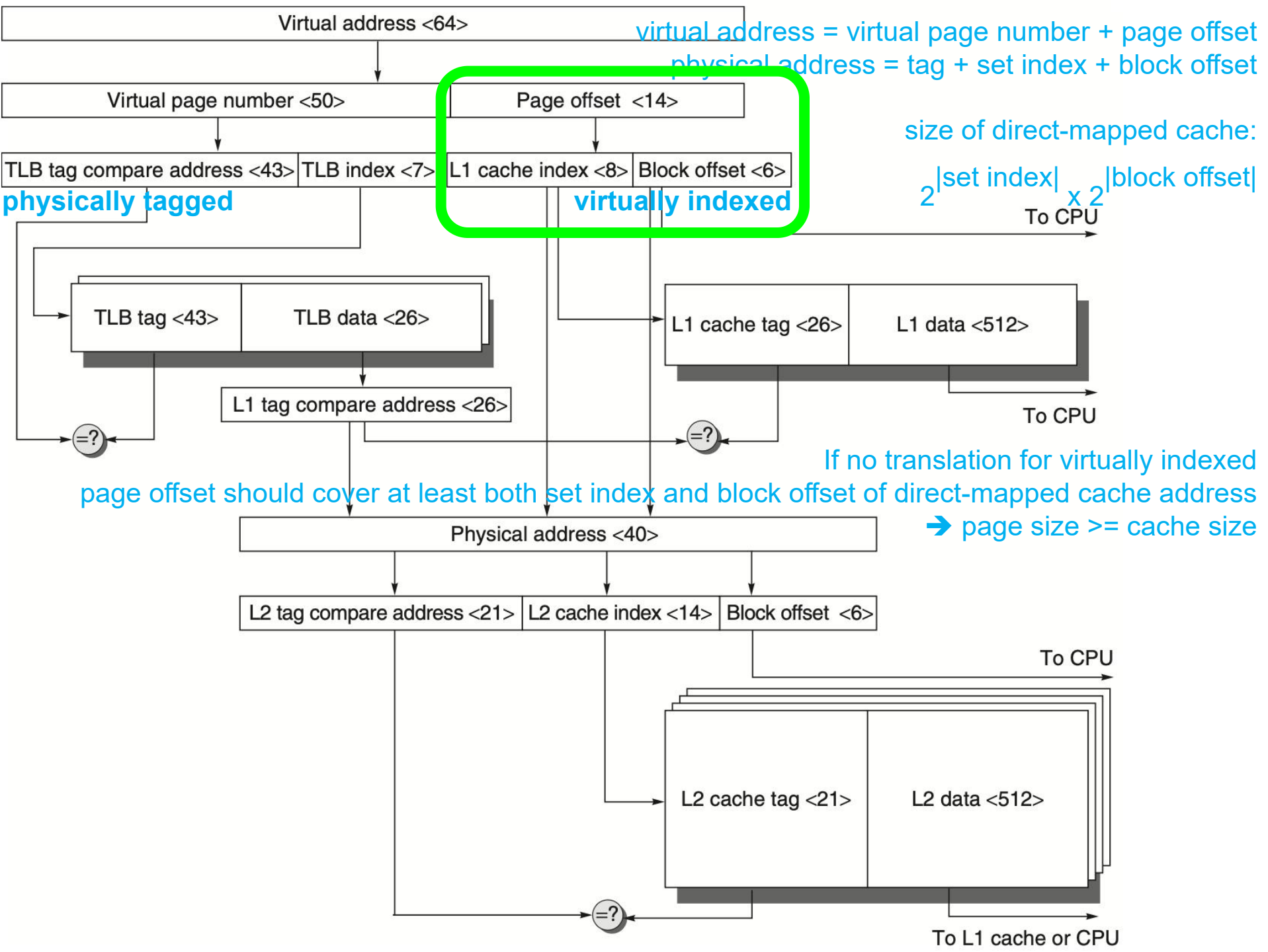
- Smaller page table, less memory (or other resources used for the memory map);
- Larger cache with fast cache hit;
- Transferring larger pages to or from secondary storage is more efficient than transferring smaller pages;
- Map more memory, reduce the number of TLB misses;

Page Size Selection

Pros of **larger** page size

- Smaller page table, less memory (or other resources used for the memory map);
- Larger cache with fast cache hit?
- Transferring larger pages to or from secondary storage is more efficient than transferring smaller pages;
- Map more memory, reduce the number of TLB misses;





Page Size Selection

Pros of **smaller** page size

- Conserve storage

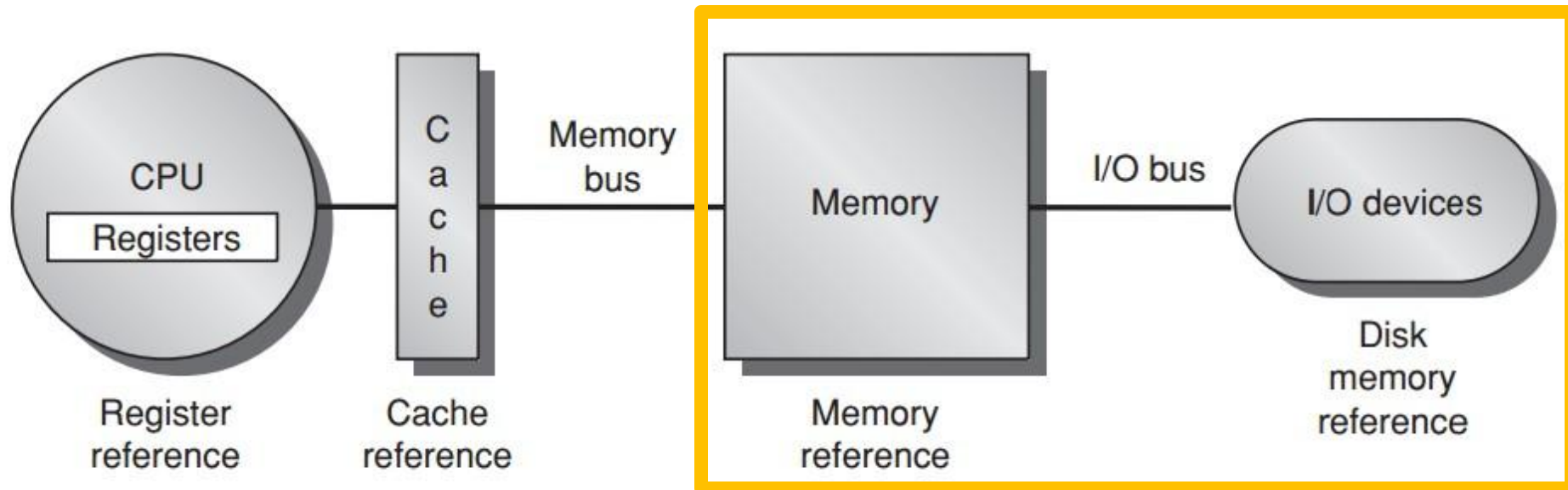
When a contiguous region of virtual memory is not equal in size to a multiple of the page size, a small page size results in less wasted storage due to internal fragmentation.

Page Size Selection

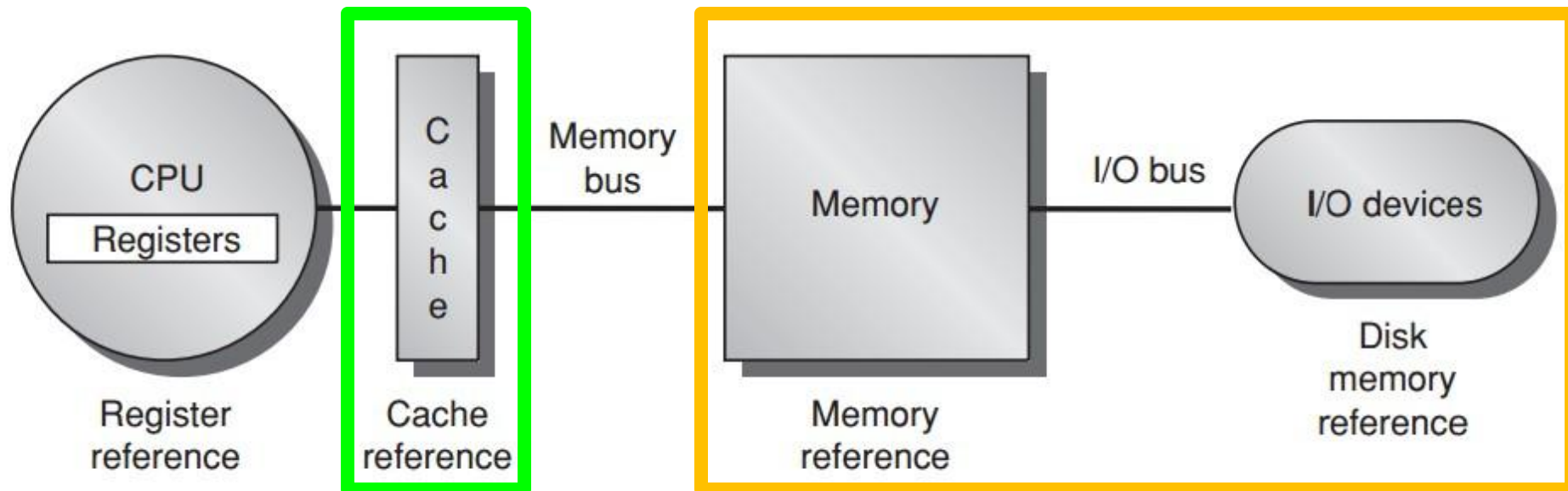
- Use both: **multiple page sizes**
- Recent microprocessors have decided to support multiple page sizes, mainly because of larger page size reduces the # of TLB entries and thus the # of TLB misses;

for some programs, TLB misses can be as significant on CPI as the cache misses;

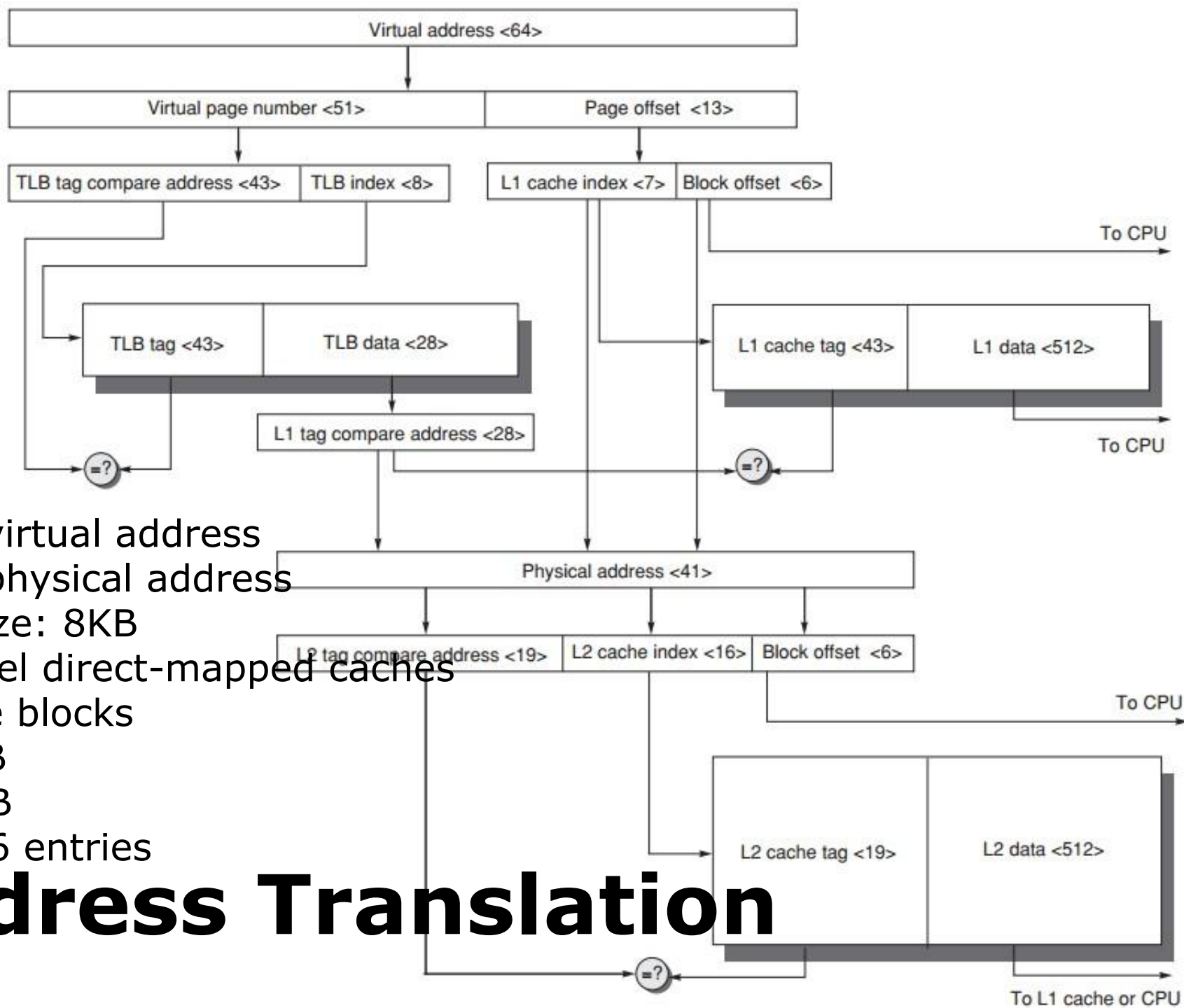
Virtual Memory = Main Memory + Secondary Storage



Virtual Memory = Main Memory + Secondary Storage

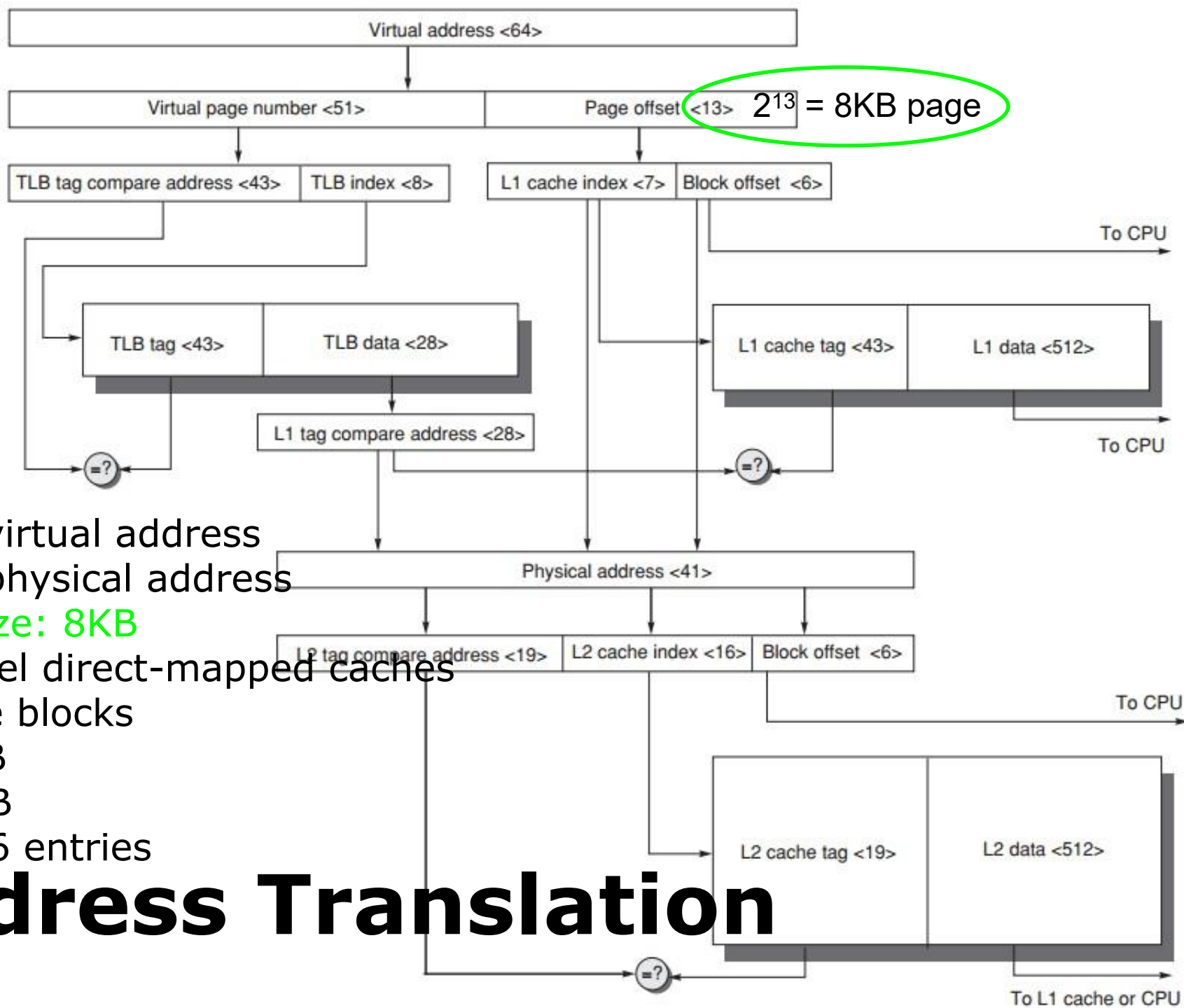


Address Translation:
one more time, with cache



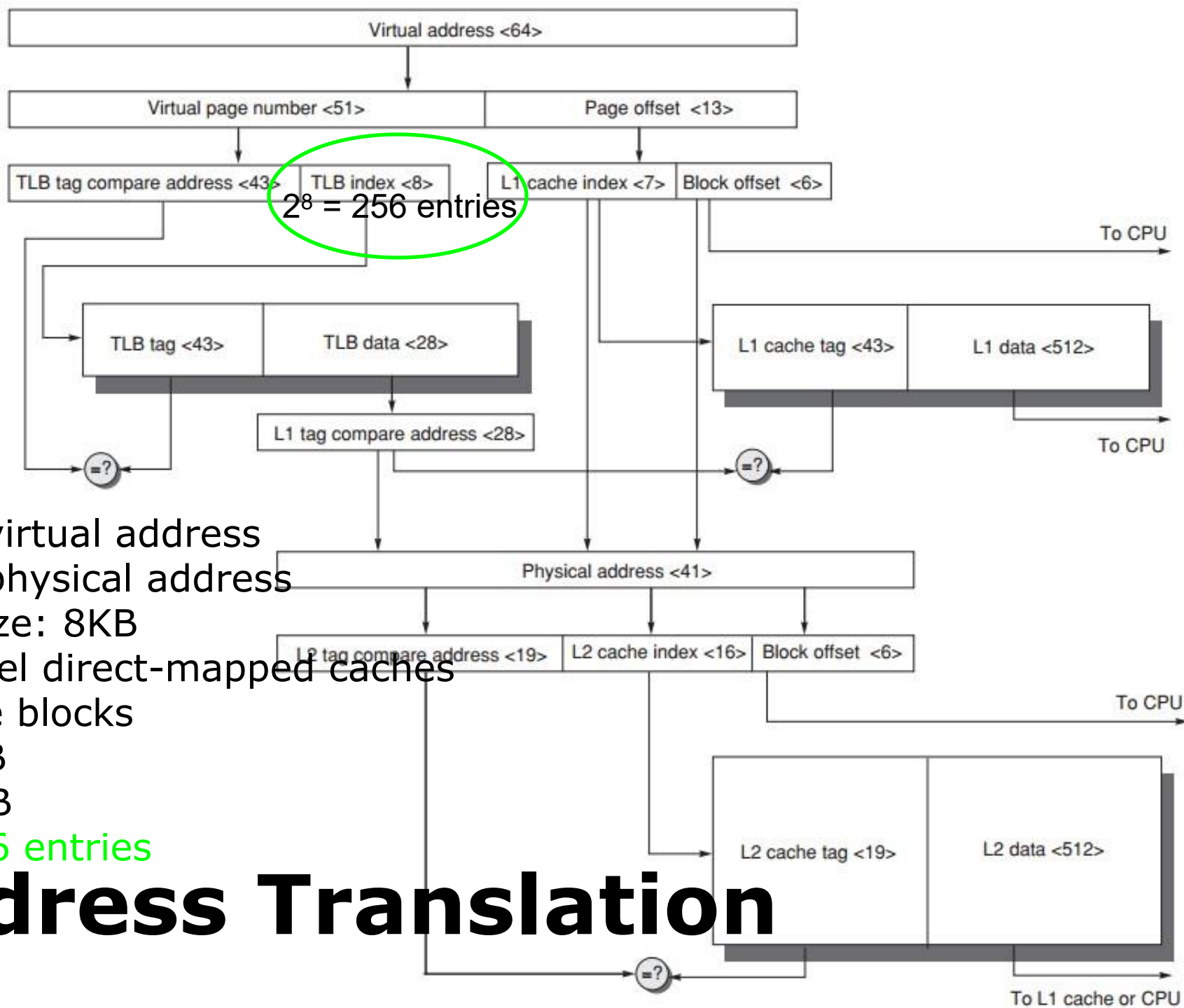
64-bit virtual address
41-bit physical address
page size: 8KB
two-level direct-mapped caches
64-byte blocks
L1: 8KB
L2: 4MB
TLB 256 entries

Address Translation



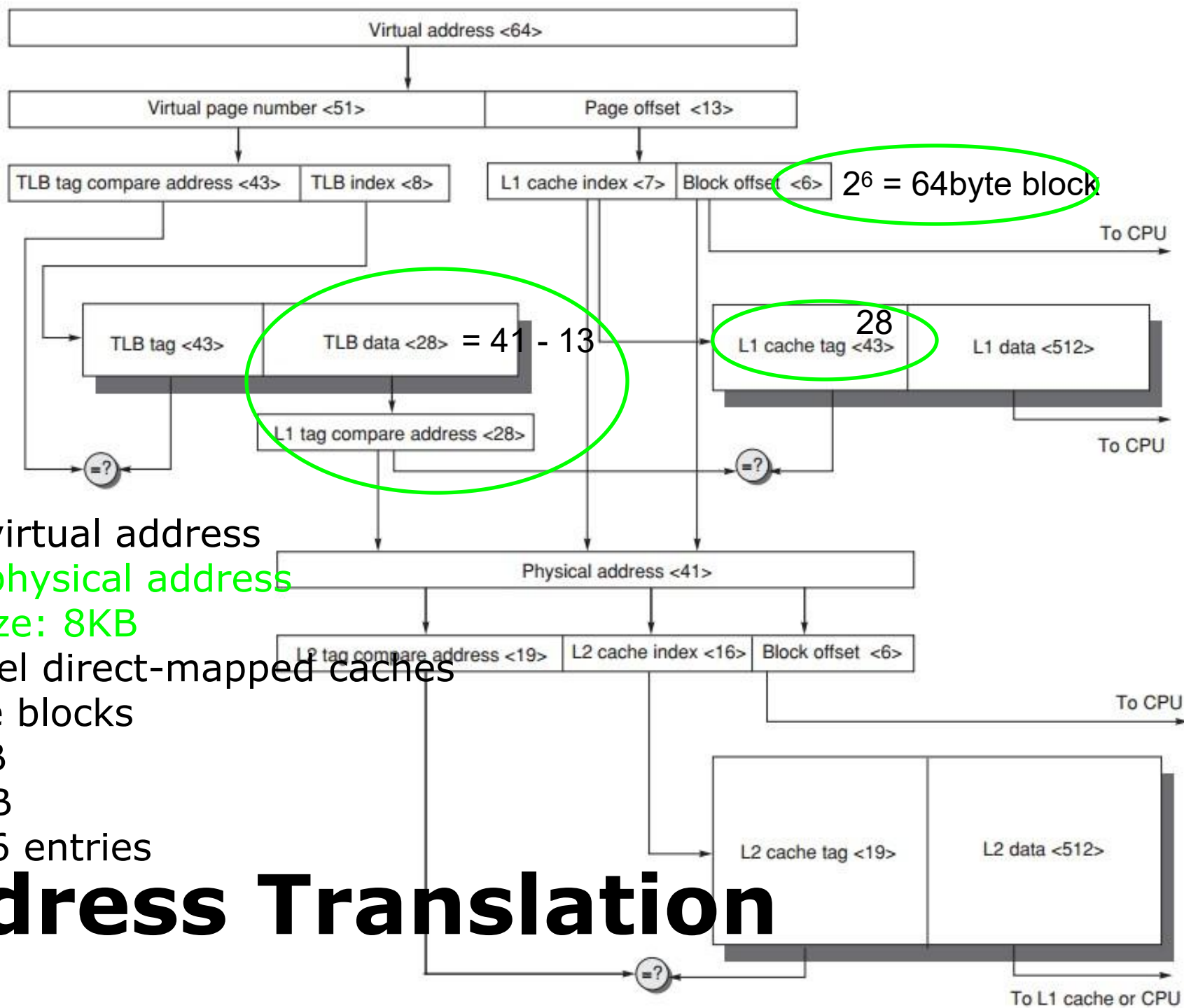
64-bit virtual address
41-bit physical address
page size: 8KB
two-level direct-mapped caches
64-byte blocks
L1: 8KB
L2: 4MB
TLB 256 entries

Address Translation



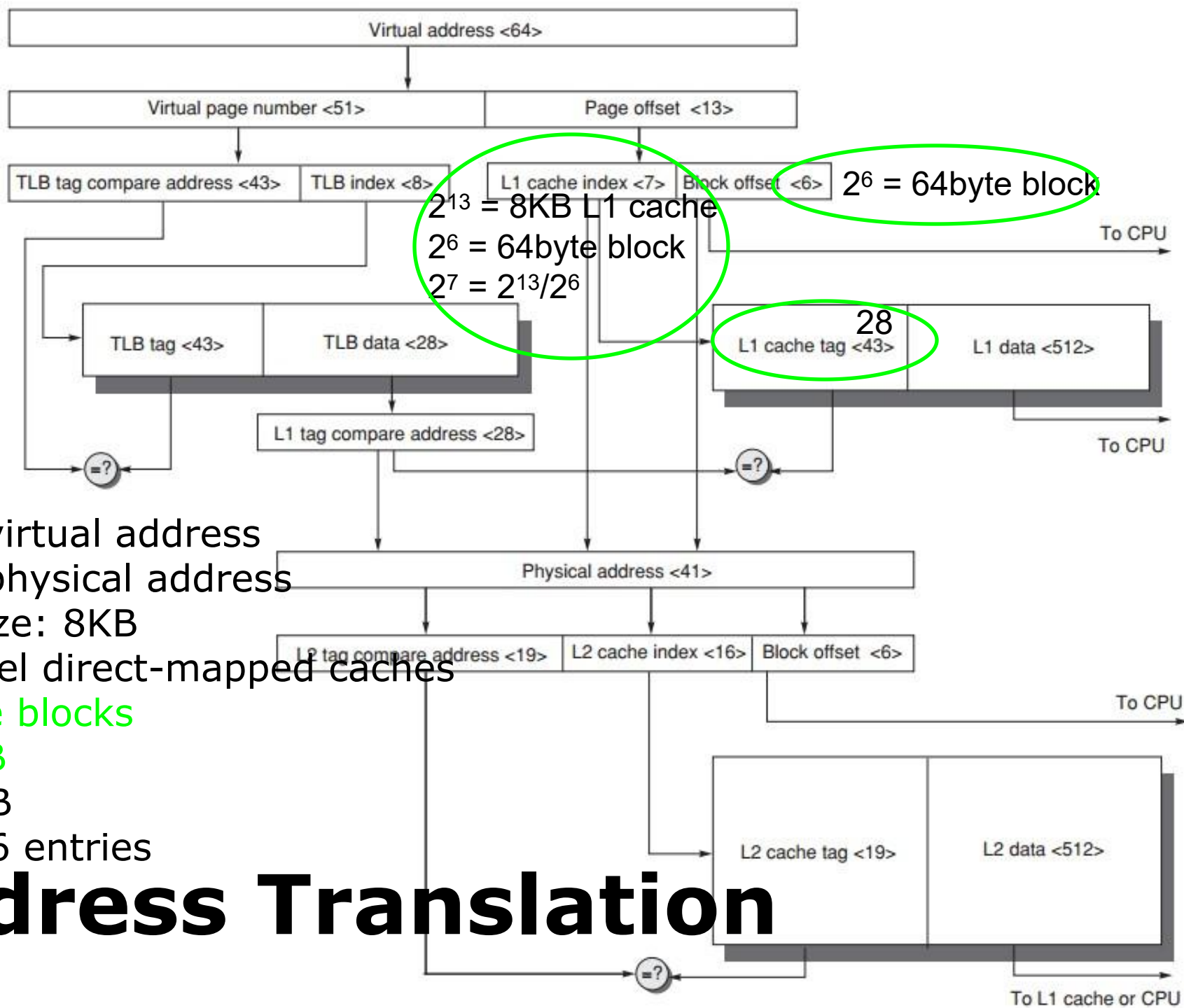
64-bit virtual address
41-bit physical address
page size: 8KB
two-level direct-mapped caches
64-byte blocks
L1: 8KB
L2: 4MB
TLB 256 entries

Address Translation



64-bit virtual address
41-bit physical address
page size: 8KB
two-level direct-mapped caches
64-byte blocks
L1: 8KB
L2: 4MB
TLB 256 entries

Address Translation



64-bit virtual address
41-bit physical address
page size: 8KB
two-level direct-mapped caches

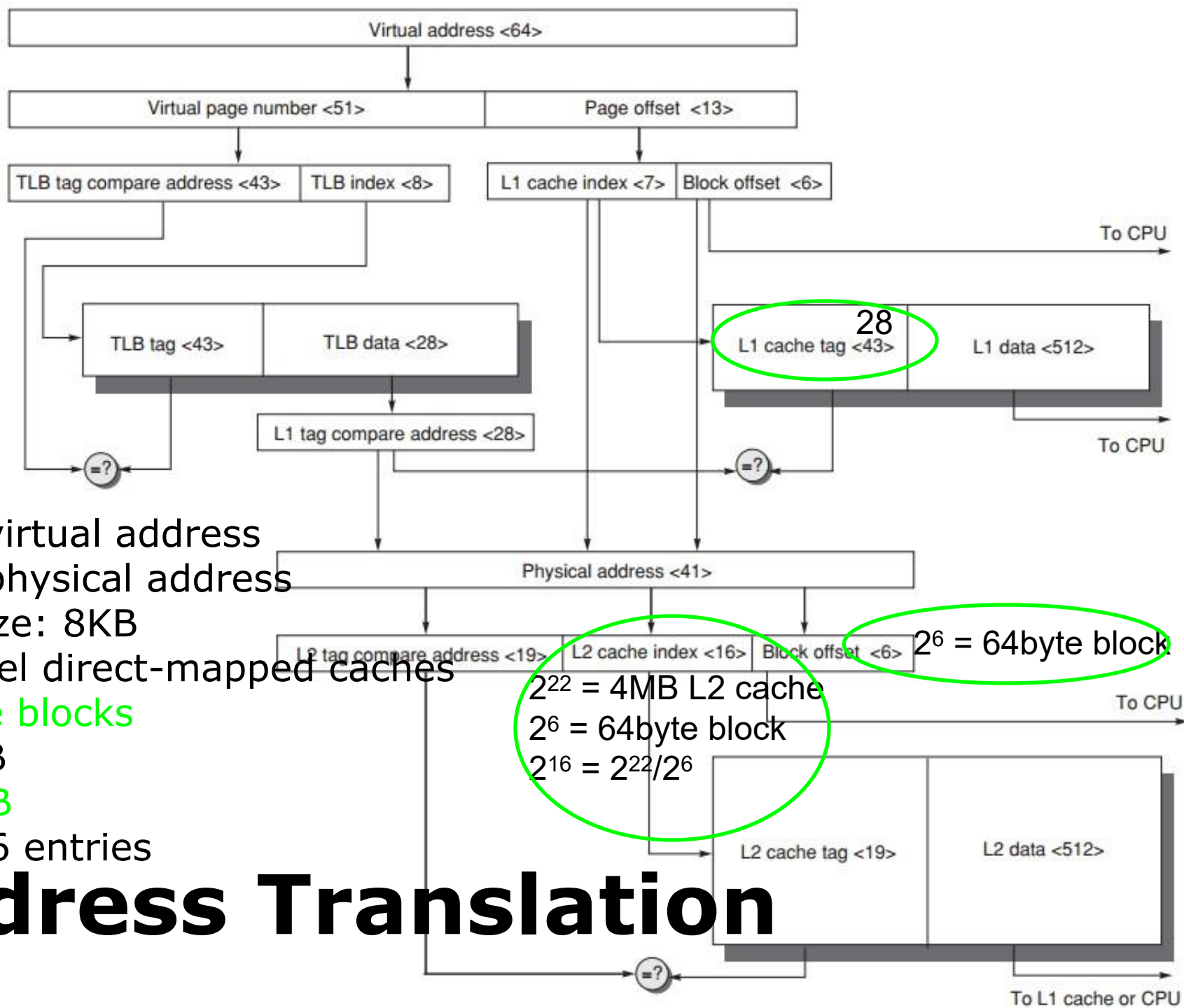
64-byte blocks

L1: 8KB

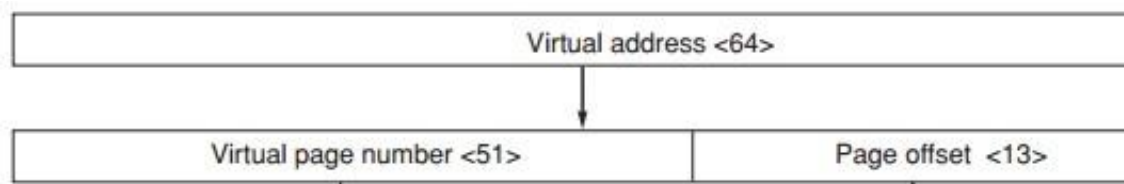
L2: 4MB

TLB 256 entries

Address Translation

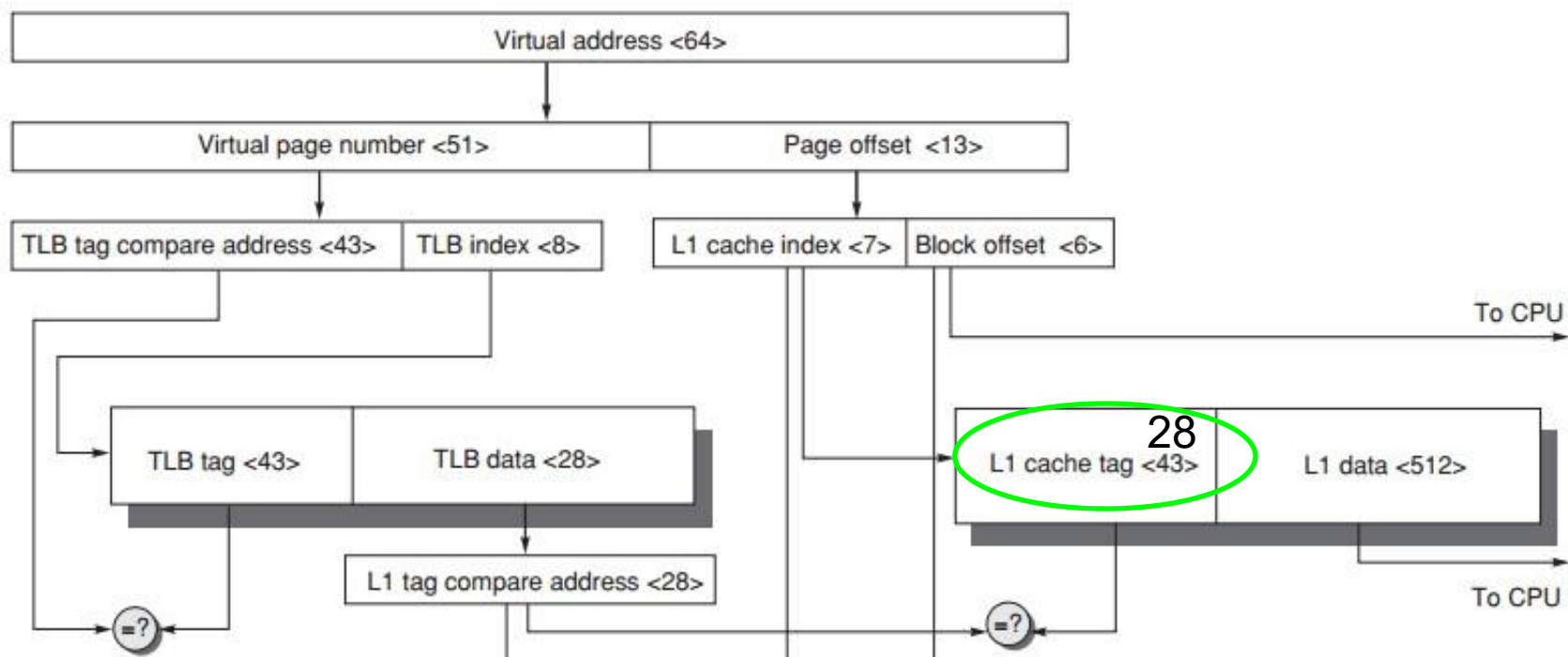


ready?



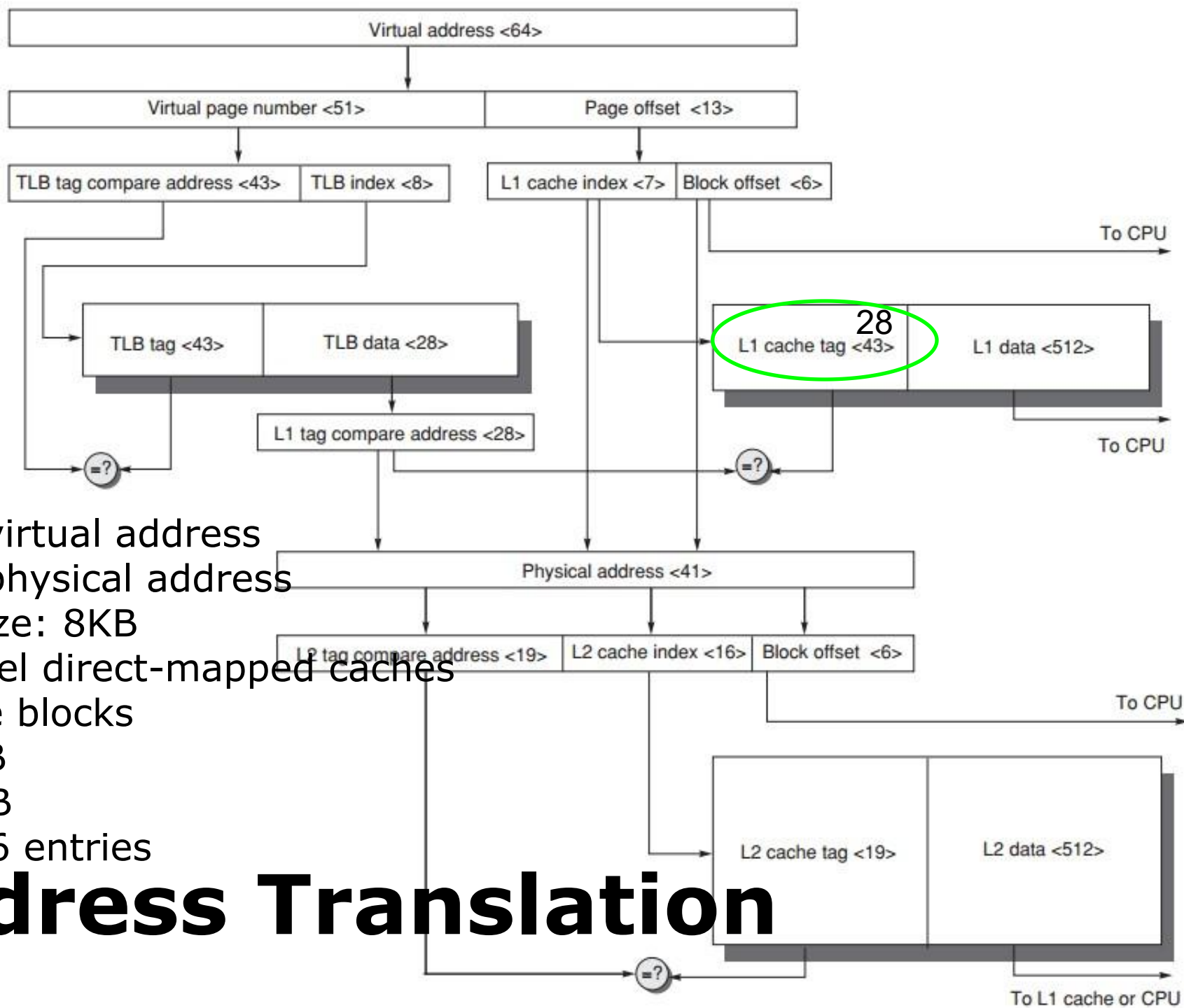
64-bit virtual address
41-bit physical address
page size: 8KB
two-level direct-mapped caches
64-byte blocks
L1: 8KB
L2: 4MB
TLB 256 entries

Address Translation



64-bit virtual address
 41-bit physical address
 page size: 8KB
 two-level direct-mapped caches
 64-byte blocks
 L1: 8KB
 L2: 4MB
 TLB 256 entries

Address Translation

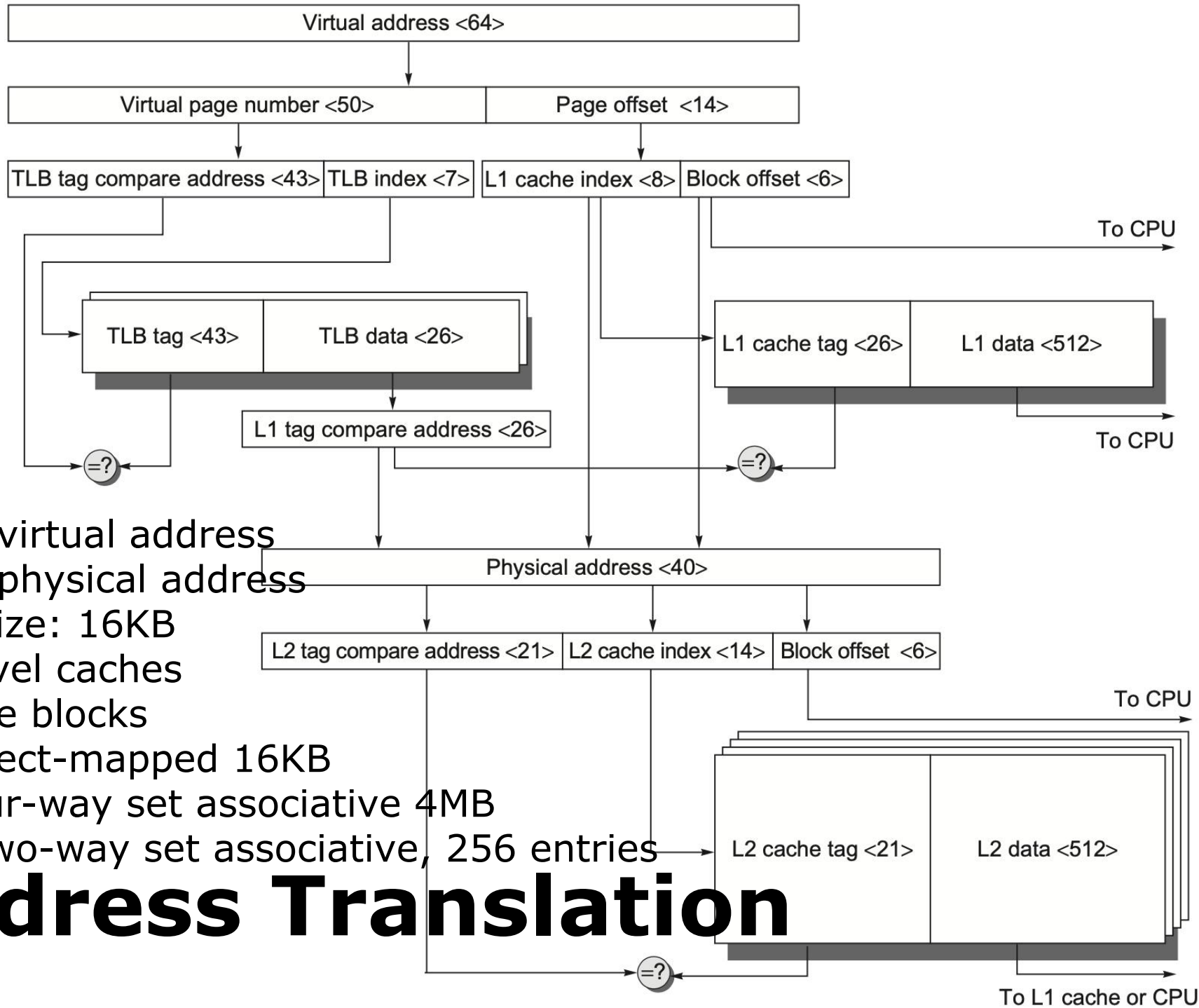


64-bit virtual address
41-bit physical address
page size: 8KB
two-level direct-mapped caches
64-byte blocks
L1: 8KB
L2: 4MB
TLB 256 entries

Address Translation

ready?

one more example, 6th ed.



Summary of Cache Opt

Technique	Hit time	Miss penalty	Miss rate	Hardware complexity	Comment
Larger block size		–	+	0	Trivial; Pentium 4 L2 uses 128 bytes
Larger cache size	–		+	1	Widely used, especially for L2 caches
Higher associativity	–		+	1	Widely used
Multilevel caches		+		2	Costly hardware; harder if L1 block size $\neq$ L2 block size; widely used
Read priority over writes		+		1	Widely used
Avoiding address translation during cache indexing	+			1	Widely used